

Cranfield CFD Assignment

1.0

Generated by Doxygen 1.15.0

1 1D Heat Equation Solver - Cranfield Assignment	1
1.1 Problem Description	1
1.2 Software Architecture (UML)	1
1.3 Build and Run Instructions	2
1.4 Group Members	2
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Class Documentation	9
5.1 CrankNicholson_Scheme Class Reference	9
5.1.1 Detailed Description	10
5.1.2 Constructor & Destructor Documentation	11
5.1.2.1 CrankNicholson_Scheme()	11
5.1.3 Member Function Documentation	12
5.1.3.1 setup_rhs_vector()	12
5.1.3.2 setup_thomas_coefficients()	12
5.1.3.3 Solve()	13
5.1.3.4 solve_step()	13
5.1.3.5 solve_thomas_algorithm()	13
5.1.4 Member Data Documentation	14
5.1.4.1 a_	14
5.1.4.2 b_	14
5.1.4.3 c_	14
5.1.4.4 d_	14
5.1.4.5 num_internal_nodes_	14
5.1.4.6 r_	15
5.1.4.7 T_solution_	15
5.2 DufortFrankel_Scheme Class Reference	15
5.2.1 Detailed Description	16
5.2.2 Constructor & Destructor Documentation	16
5.2.2.1 DufortFrankel_Scheme()	16
5.2.3 Member Function Documentation	17
5.2.3.1 dfSolution()	17
5.2.4 Member Data Documentation	17
5.2.4.1 problem	17
5.2.4.2 r	18
5.3 Laasonen_Scheme Class Reference	18

5.3.1 Detailed Description	19
5.3.2 Constructor & Destructor Documentation	19
5.3.2.1 Laasonen_Scheme()	19
5.3.3 Member Function Documentation	20
5.3.3.1 setup_rhs_vector()	20
5.3.3.2 setup_thomas_coefficients()	20
5.3.3.3 Solve()	20
5.3.3.4 solve_step()	20
5.3.3.5 solve_thomas_algorithm()	21
5.3.4 Member Data Documentation	21
5.3.4.1 a_	21
5.3.4.2 b_	21
5.3.4.3 c_	22
5.3.4.4 d_	22
5.3.4.5 num_internal_nodes_	22
5.3.4.6 r_	22
5.3.4.7 T_solution_	22
5.4 Output Class Reference	23
5.4.1 Detailed Description	23
5.4.2 Constructor & Destructor Documentation	23
5.4.2.1 Output() [1/2]	23
5.4.2.2 Output() [2/2]	23
5.4.3 Member Function Documentation	24
5.4.3.1 Generate_Diagram()	24
5.4.3.2 Generate_File()	24
5.4.3.3 Get_filename()	25
5.4.3.4 Set_filename()	25
5.4.4 Member Data Documentation	25
5.4.4.1 filename	25
5.5 ProblemDefinition Class Reference	26
5.5.1 Detailed Description	27
5.5.2 Constructor & Destructor Documentation	27
5.5.2.1 ProblemDefinition() [1/2]	27
5.5.2.2 ProblemDefinition() [2/2]	27
5.5.3 Member Function Documentation	28
5.5.3.1 Analytic_Solution()	28
5.5.3.2 Get_D()	28
5.5.3.3 Get_dt()	29
5.5.3.4 Get_dx()	29
5.5.3.5 Get_N()	29
5.5.3.6 Get_thickness()	30
5.5.3.7 Get_Tin()	30

5.5.3.8 Get_tMax()	30
5.5.3.9 Get_Tsur()	30
5.5.3.10 Set_D()	30
5.5.3.11 Set_dt()	31
5.5.3.12 Set_dx()	31
5.5.3.13 Set_N()	31
5.5.3.14 Set_thickness()	32
5.5.3.15 Set_Tin()	32
5.5.3.16 Set_tMax()	32
5.5.3.17 Set_Tsur()	33
5.5.4 Member Data Documentation	34
5.5.4.1 D	34
5.5.4.2 dt	34
5.5.4.3 dx	34
5.5.4.4 N	34
5.5.4.5 thickness	35
5.5.4.6 Tin	35
5.5.4.7 tMax	35
5.5.4.8 Tsur	35
5.6 Richardson_Scheme Class Reference	35
5.6.1 Detailed Description	36
5.6.2 Constructor & Destructor Documentation	37
5.6.2.1 Richardson_Scheme()	37
5.6.3 Member Function Documentation	38
5.6.3.1 richardsonSolution()	38
5.6.4 Member Data Documentation	38
5.6.4.1 problem	38
5.6.4.2 r	39
5.7 Scheme Class Reference	39
5.7.1 Detailed Description	40
5.7.2 Constructor & Destructor Documentation	40
5.7.2.1 Scheme() [1/2]	40
5.7.2.2 Scheme() [2/2]	40
5.7.3 Member Function Documentation	41
5.7.3.1 Get_CFL()	41
5.7.3.2 Get_D()	41
5.7.3.3 Get_dt()	42
5.7.3.4 Get_dx()	42
5.7.3.5 Get_N()	42
5.7.3.6 Set_D()	42
5.7.3.7 Set_dt()	43
5.7.3.8 Set_dx()	43

5.7.3.9 Set_N()	43
5.7.4 Member Data Documentation	43
5.7.4.1 D	43
5.7.4.2 dt	43
5.7.4.3 dx	44
5.7.4.4 N	44
5.7.4.5 r	44
5.8 Verification Class Reference	44
5.8.1 Detailed Description	45
5.8.2 Constructor & Destructor Documentation	45
5.8.2.1 Verification() [1/2]	45
5.8.2.2 Verification() [2/2]	45
5.8.3 Member Function Documentation	46
5.8.3.1 Calculation_L2_Norm()	46
5.8.3.2 Calculation_Relative_Error()	46
5.8.3.3 Get_CFL()	47
5.8.3.4 Set_CFL()	47
5.8.3.5 Verify_Stability()	47
5.8.4 Member Data Documentation	48
5.8.4.1 CFL	48
6 File Documentation	49
6.1 CrankNicholson_Scheme.cpp File Reference	49
6.2 CrankNicholson_Scheme.cpp	49
6.3 CrankNicholson_Scheme.h File Reference	51
6.4 CrankNicholson_Scheme.h	51
6.5 DufortFrankel_Scheme.cpp File Reference	52
6.6 DufortFrankel_Scheme.cpp	52
6.7 DufortFrankel_Scheme.h File Reference	53
6.8 DufortFrankel_Scheme.h	54
6.9 Laasonen_Scheme.cpp File Reference	54
6.10 Laasonen_Scheme.cpp	54
6.11 Laasonen_Scheme.h File Reference	56
6.12 Laasonen_Scheme.h	57
6.13 main.cpp File Reference	57
6.13.1 Function Documentation	58
6.13.1.1 main()	58
6.14 main.cpp	58
6.15 Output.cpp File Reference	61
6.16 Output.cpp	61
6.17 Output.h File Reference	63
6.18 Output.h	63

6.19 ProblemDefinition.cpp File Reference	63
6.19.1 Macro Definition Documentation	64
6.19.1.1 M_PI	64
6.20 ProblemDefinition.cpp	64
6.21 ProblemDefinition.h File Reference	66
6.22 ProblemDefinition.h	66
6.23 README.md File Reference	67
6.24 Richardson_Scheme.cpp File Reference	67
6.25 Richardson_Scheme.cpp	67
6.26 Richardson_Scheme.h File Reference	69
6.27 Richardson_Scheme.h	69
6.28 Scheme.cpp File Reference	69
6.29 Scheme.cpp	70
6.30 Scheme.h File Reference	71
6.31 Scheme.h	71
6.32 Verification.cpp File Reference	71
6.33 Verification.cpp	72
6.34 Verification.h File Reference	73
6.35 Verification.h	73
Index	75

Chapter 1

1D Heat Equation Solver - Cranfield Assignment

This project is an object-oriented C++ implementation for solving the one-dimensional heat equation using various finite difference methods, as part of the Cranfield University "Computational Methods & C++" assignment.

1.1 Problem Description

The project aims to compute the temperature distribution over time within a 31cm thick wall.

- **Governing Equation:** $\frac{\partial T}{\partial t} = D \frac{\partial^2 T}{\partial x^2}$
- **Physical Properties:**
 - Wall Thickness (L): 31 cm
 - Thermal Diffusivity (D): 93 cm²/hr
 - Uniform Initial Temperature (T_{in}): 38 °C
 - Maintained Surface Temperature (T_{sur}): 149 °C
- **Numerical Methods to Implement:**
 1. DuFort-Frankel (Explicit)
 2. Richardson (Explicit)
 3. Laasonen (Implicit)
 4. Crank-Nicholson (Implicit)

1.2 Software Architecture (UML)

The software design allows for modularity and extensibility. Below is the UML class diagram representing the project structure.

Detailed caller and collaboration graphs are available in the generated Doxygen documentation (see <docs/html/index.html>).

Key Classes:

- **ProblemDefinition**: Encapsulates all physical and numerical parameters of the problem (diffusivity, dimensions, grid spacing, etc.).
- **Scheme (Base Class)**: An abstract base class defining the common interface (`Solve()`) for all numerical methods.
- **DufortFrankel_Scheme, Richardson_Scheme, Laasonen_Scheme, CrankNicholson_Scheme**↔ : Concrete implementations of each numerical scheme, inheriting from `Scheme`.
- **Output**: Handles data export to CSV files for analysis.
- **Verification**: Provides utilities for verifying stability (L2 Norm, Relative Error calculation) and comparing numerical results against the analytical solution.

1.3 Build and Run Instructions

1. **Prerequisites**: A C++ compiler (supporting C++17 or higher), such as g++.
2. **Clone the repository**: `bash git clone <your-repo-url> cd <your-repo-name>`
3. **Compile the project**: `bash g++ -Wall *.cpp -o main`
4. **Run the program**: `bash ./main`

1.4 Group Members

- Clémence-Philomène Hinot
- John Hoarau

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Output	23
ProblemDefinition	26
Scheme	39
CrankNicholson_Scheme	9
DufortFrankel_Scheme	15
Laasonen_Scheme	18
Richardson_Scheme	35
Verification	44

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

CrankNicholson_Scheme	Implements the Crank-Nicholson implicit numerical scheme	9
DufortFrankel_Scheme	Implements the Dufort-Frankel explicit numerical scheme	15
Laasonen_Scheme	Implements the Laasonen implicit numerical scheme	18
Output	Handles data export and visualization	23
ProblemDefinition		26
Richardson_Scheme	Implements the Richardson explicit numerical scheme	35
Scheme	Base class for numerical schemes	39
Verification	Provides methods for verifying numerical results and conditions	44

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

CrankNicolson_Scheme.cpp	49
CrankNicolson_Scheme.h	51
DufortFrankel_Scheme.cpp	52
DufortFrankel_Scheme.h	53
Laasonen_Scheme.cpp	54
Laasonen_Scheme.h	56
main.cpp	57
Output.cpp	61
Output.h	63
ProblemDefinition.cpp	63
ProblemDefinition.h	66
Richardson_Scheme.cpp	67
Richardson_Scheme.h	69
Scheme.cpp	69
Scheme.h	71
Verification.cpp	71
Verification.h	73

Chapter 5

Class Documentation

5.1 CrankNicholson_Scheme Class Reference

Implements the Crank-Nicholson implicit numerical scheme.

```
#include <CrankNicholson_Scheme.h>
```

Inheritance diagram for CrankNicholson_Scheme:

Collaboration diagram for CrankNicholson_Scheme:

Public Member Functions

- [CrankNicholson_Scheme](#) (const [ProblemDefinition](#) &problem)
Constructor.
- [vector< vector< double > >](#) [Solve](#) ()
Solves the heat equation using the Crank-Nicholson scheme.
- void [solve_step](#) (std::vector< double > &T_current, double T_sur)
Performs a single time step of the Crank-Nicholson scheme.

Public Member Functions inherited from [Scheme](#)

- [Scheme](#) ()
Default constructor.
- [Scheme](#) (double [dx](#), int [N](#), double [dt](#), double [D](#))
Parameterized constructor.
- double [Get_dx](#) () const
Get spatial grid spacing.
- int [Get_N](#) () const
Get Fourier Number.
- double [Get_dt](#) () const
Get time step size.
- double [Get_D](#) () const
Get thermal diffusivity.
- void [Set_dx](#) (double value)

- *Set spatial grid spacing.*
- void [Set_N](#) (int value)
- *Set Fourier Number.*
- void [Set_dt](#) (double value)
- *Set time step size.*
- void [Set_D](#) (double value)
- *Set thermal diffusivity.*
- double [Get_CFL](#) ()
- *Calculates and returns the CFL number.*

Private Member Functions

- void [setup_thomas_coefficients](#) ()
- *Sets up the constant coefficients of the tridiagonal matrix. Crank-Nicholson: $a = -r/2$, $b = 1+r$, $c = -r/2$.*
- void [setup_rhs_vector](#) (const std::vector< double > &T_current, double T_sur)
- *Computes the RHS vector 'd'. Crank-Nicholson RHS involves T^n terms: $d_i = (r/2)T_{i-1}^n + (1-r)T_i^n + (r/2)T_{i+1}^n$.*
- void [solve_thomas_algorithm](#) ()
- *Solves the tridiagonal system using Thomas algorithm. Updates T_solution_ with the new temperature values.*

Private Attributes

- double [r_](#)
- *CFL number.*
- int [num_internal_nodes_](#)
- *Number of internal nodes (N-1).*
- std::vector< double > [a_](#)
- *Sub-diagonal elements.*
- std::vector< double > [b_](#)
- *Main diagonal elements.*
- std::vector< double > [c_](#)
- *Super-diagonal elements.*
- std::vector< double > [d_](#)
- *Right-hand side vector.*
- std::vector< double > [T_solution_](#)
- *Temporary vector to store the solution from Thomas algorithm.*

5.1.1 Detailed Description

Implements the Crank-Nicholson implicit numerical scheme.

The Crank-Nicholson scheme is a semi-implicit method (average of forward and backward time steps). It is unconditionally stable and second-order accurate in both time and space. It requires solving a tridiagonal system of linear equations at each time step.

Definition at line 30 of file [CrankNicholson_Scheme.h](#).

5.1.2 Constructor & Destructor Documentation

5.1.2.1 CrankNicholson_Scheme()

```
CrankNicholson_Scheme::CrankNicholson_Scheme (  
    const ProblemDefinition & problem)
```

Constructor.

Parameters

<i>problem</i>	The problem definition containing physical parameters.
----------------	--

Definition at line 21 of file [CrankNicholson_Scheme.cpp](#).

References [a_](#), [b_](#), [c_](#), [d_](#), [Scheme::Get_D\(\)](#), [Scheme::Get_dt\(\)](#), [ProblemDefinition::Get_dx\(\)](#), [Scheme::Get_dx\(\)](#), [Scheme::Get_N\(\)](#), [ProblemDefinition::Get_thickness\(\)](#), [num_internal_nodes_](#), [r_](#), [Scheme::Scheme\(\)](#), [setup_thomas_coefficients\(\)](#), and [T_solution_](#).

Here is the call graph for this function:

5.1.3 Member Function Documentation

5.1.3.1 `setup_rhs_vector()`

```
void CrankNicholson_Scheme::setup_rhs_vector (
    const std::vector< double > & T_current,
    double T_sur) [private]
```

Computes the RHS vector 'd'. Crank-Nicholson RHS involves T^n terms: $d_i = (r/2)T_{i-1}^n + (1-r)T_i^n + (r/2)T_{i+1}^n$.

Parameters

<i>T_current</i>	The temperature distribution at the current time step.
<i>T_sur</i>	The surface temperature (boundary condition).

Definition at line 63 of file [CrankNicholson_Scheme.cpp](#).

References [d_](#), [num_internal_nodes_](#), and [r_](#).

Referenced by [solve_step\(\)](#).

Here is the caller graph for this function:

5.1.3.2 `setup_thomas_coefficients()`

```
void CrankNicholson_Scheme::setup_thomas_coefficients () [private]
```

Sets up the constant coefficients of the tridiagonal matrix. Crank-Nicholson: $a = -r/2$, $b = 1+r$, $c = -r/2$.

Definition at line 50 of file [CrankNicholson_Scheme.cpp](#).

References [a_](#), [b_](#), [c_](#), [num_internal_nodes_](#), and [r_](#).

Referenced by [CrankNicholson_Scheme\(\)](#).

Here is the caller graph for this function:

5.1.3.3 Solve()

```
vector< vector< double > > CrankNicholson_Scheme::Solve ()
```

Solves the heat equation using the Crank-Nicholson scheme.

Returns

A 2D vector containing the temperature distribution over time and space.

Definition at line 135 of file [CrankNicholson_Scheme.cpp](#).

References [Scheme::Get_dx\(\)](#), [Scheme::Get_N\(\)](#), and [solve_step\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.1.3.4 solve_step()

```
void CrankNicholson_Scheme::solve_step (
    std::vector< double > & T_current,
    double T_sur)
```

Performs a single time step of the Crank-Nicholson scheme.

Parameters

<i>T_current</i>	The current temperature distribution (input/output).
<i>T_sur</i>	The surface temperature (boundary condition).

Definition at line 124 of file [CrankNicholson_Scheme.cpp](#).

References [num_internal_nodes_](#), [setup_rhs_vector\(\)](#), [solve_thomas_algorithm\(\)](#), and [T_solution_](#).

Referenced by [Solve\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.1.3.5 solve_thomas_algorithm()

```
void CrankNicholson_Scheme::solve_thomas_algorithm () [private]
```

Solves the tridiagonal system using Thomas algorithm. Updates [T_solution_](#) with the new temperature values.

Definition at line 94 of file [CrankNicholson_Scheme.cpp](#).

References [a_](#), [b_](#), [c_](#), [d_](#), [num_internal_nodes_](#), and [T_solution_](#).

Referenced by [solve_step\(\)](#).

Here is the caller graph for this function:

5.1.4 Member Data Documentation

5.1.4.1 `a_`

```
std::vector<double> CrankNicholson_Scheme::a_ [private]
```

Sub-diagonal elements.

Definition at line 37 of file [CrankNicholson_Scheme.h](#).

Referenced by [CrankNicholson_Scheme\(\)](#), [setup_thomas_coefficients\(\)](#), and [solve_thomas_algorithm\(\)](#).

5.1.4.2 `b_`

```
std::vector<double> CrankNicholson_Scheme::b_ [private]
```

Main diagonal elements.

Definition at line 38 of file [CrankNicholson_Scheme.h](#).

Referenced by [CrankNicholson_Scheme\(\)](#), [setup_thomas_coefficients\(\)](#), and [solve_thomas_algorithm\(\)](#).

5.1.4.3 `c_`

```
std::vector<double> CrankNicholson_Scheme::c_ [private]
```

Super-diagonal elements.

Definition at line 39 of file [CrankNicholson_Scheme.h](#).

Referenced by [CrankNicholson_Scheme\(\)](#), [setup_thomas_coefficients\(\)](#), and [solve_thomas_algorithm\(\)](#).

5.1.4.4 `d_`

```
std::vector<double> CrankNicholson_Scheme::d_ [private]
```

Right-hand side vector.

Definition at line 41 of file [CrankNicholson_Scheme.h](#).

Referenced by [CrankNicholson_Scheme\(\)](#), [setup_rhs_vector\(\)](#), and [solve_thomas_algorithm\(\)](#).

5.1.4.5 `num_internal_nodes_`

```
int CrankNicholson_Scheme::num_internal_nodes_ [private]
```

Number of internal nodes (N-1).

Definition at line 34 of file [CrankNicholson_Scheme.h](#).

Referenced by [CrankNicholson_Scheme\(\)](#), [setup_rhs_vector\(\)](#), [setup_thomas_coefficients\(\)](#), [solve_step\(\)](#), and [solve_thomas_algorithm\(\)](#).

5.1.4.6 `r_`

```
double CrankNicholson_Scheme::r_ [private]
```

CFL number.

Definition at line 33 of file [CrankNicholson_Scheme.h](#).

Referenced by [CrankNicholson_Scheme\(\)](#), [setup_rhs_vector\(\)](#), and [setup_thomas_coefficients\(\)](#).

5.1.4.7 `T_solution_`

```
std::vector<double> CrankNicholson_Scheme::T_solution_ [private]
```

Temporary vector to store the solution from Thomas algorithm.

Definition at line 43 of file [CrankNicholson_Scheme.h](#).

Referenced by [CrankNicholson_Scheme\(\)](#), [solve_step\(\)](#), and [solve_thomas_algorithm\(\)](#).

The documentation for this class was generated from the following files:

- [CrankNicholson_Scheme.h](#)
- [CrankNicholson_Scheme.cpp](#)

5.2 DufortFrankel_Scheme Class Reference

Implements the Dufort-Frankel explicit numerical scheme.

```
#include <DufortFrankel_Scheme.h>
```

Inheritance diagram for DufortFrankel_Scheme:

Collaboration diagram for DufortFrankel_Scheme:

Public Member Functions

- [DufortFrankel_Scheme](#) (const [ProblemDefinition](#) &problem)
Constructor.
- `vector< vector< double > > dfSolution` (vector< vector< double > > laasonen)
Solves the heat equation using the Dufort-Frankel scheme.

Public Member Functions inherited from [Scheme](#)

- [Scheme](#) ()
Default constructor.
- [Scheme](#) (double [dx](#), int [N](#), double [dt](#), double [D](#))
Parameterized constructor.
- double [Get_dx](#) () const
Get spatial grid spacing.
- int [Get_N](#) () const
Get Fourier Number.
- double [Get_dt](#) () const
Get time step size.
- double [Get_D](#) () const
Get thermal diffusivity.
- void [Set_dx](#) (double value)
Set spatial grid spacing.
- void [Set_N](#) (int value)
Set Fourier Number.
- void [Set_dt](#) (double value)
Set time step size.
- void [Set_D](#) (double value)
Set thermal diffusivity.
- double [Get_CFL](#) ()
Calculates and returns the CFL number.

Private Attributes

- double [r](#) = 0.0
CFL number.
- [ProblemDefinition](#) [problem](#)
Problem definition object.

5.2.1 Detailed Description

Implements the Dufort-Frankel explicit numerical scheme.

The Dufort-Frankel scheme is an unconditionally stable explicit method. It is a modification of the Richardson scheme to improve stability.

Definition at line 28 of file [DufortFrankel_Scheme.h](#).

5.2.2 Constructor & Destructor Documentation

5.2.2.1 DufortFrankel_Scheme()

```
DufortFrankel_Scheme::DufortFrankel_Scheme (
    const ProblemDefinition & problem)
```

Constructor.

Parameters

<i>problem</i>	The problem definition containing physical parameters.
----------------	--

Definition at line 21 of file [DufortFrankel_Scheme.cpp](#).

References [Scheme::Get_CFL\(\)](#), [Scheme::Get_D\(\)](#), [Scheme::Get_dt\(\)](#), [Scheme::Get_dx\(\)](#), [Scheme::Get_N\(\)](#), [problem](#), [r](#), and [Scheme::Scheme\(\)](#).

Here is the call graph for this function:

5.2.3 Member Function Documentation

5.2.3.1 dfSolution()

```
vector< vector< double > > DufortFrankel_Scheme::dfSolution (
    vector< vector< double > > laasonen)
```

Solves the heat equation using the Dufort-Frankel scheme.

Parameters

<i>laasonen</i>	The solution from the Laasonen scheme (used for the first time step).
-----------------	---

Returns

A 2D vector containing the temperature distribution over time and space.

Definition at line 27 of file [DufortFrankel_Scheme.cpp](#).

References [problem](#), and [r](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:

5.2.4 Member Data Documentation

5.2.4.1 problem

```
ProblemDefinition DufortFrankel_Scheme::problem [private]
```

Problem definition object.

Definition at line 32 of file [DufortFrankel_Scheme.h](#).

Referenced by [dfSolution\(\)](#), and [DufortFrankel_Scheme\(\)](#).

5.2.4.2 r

```
double DufortFrankel_Scheme::r = 0.0 [private]
```

CFL number.

Definition at line 31 of file [DufortFrankel_Scheme.h](#).

Referenced by [dfSolution\(\)](#), and [DufortFrankel_Scheme\(\)](#).

The documentation for this class was generated from the following files:

- [DufortFrankel_Scheme.h](#)
- [DufortFrankel_Scheme.cpp](#)

5.3 Laasonen_Scheme Class Reference

Implements the Laasonen implicit numerical scheme.

```
#include <Laasonen_Scheme.h>
```

Inheritance diagram for Laasonen_Scheme:

Collaboration diagram for Laasonen_Scheme:

Public Member Functions

- [Laasonen_Scheme](#) (const [ProblemDefinition](#) &problem)
Constructor.
- [vector< vector< double > > Solve](#) ()
Solves the heat equation using the Laasonen scheme.
- void [solve_step](#) (std::vector< double > &T_current, double T_sur)
Performs a single time step of the Laasonen scheme.

Public Member Functions inherited from [Scheme](#)

- [Scheme](#) ()
Default constructor.
- [Scheme](#) (double dx, int N, double dt, double D)
Parameterized constructor.
- double [Get_dx](#) () const
Get spatial grid spacing.
- int [Get_N](#) () const
Get Fourier Number.
- double [Get_dt](#) () const
Get time step size.
- double [Get_D](#) () const
Get thermal diffusivity.
- void [Set_dx](#) (double value)
Set spatial grid spacing.
- void [Set_N](#) (int value)
Set Fourier Number.
- void [Set_dt](#) (double value)
Set time step size.
- void [Set_D](#) (double value)
Set thermal diffusivity.
- double [Get_CFL](#) ()
Calculates and returns the CFL number.

Private Member Functions

- void [setup_thomas_coefficients](#) ()
Sets up the constant coefficients of the tridiagonal matrix. Laasonen: $a = -r$, $b = 1+2r$, $c = -r$.
- void [setup_rhs_vector](#) (const std::vector< double > &T_current, double T_sur)
Computes the RHS vector 'd'. Laasonen RHS is simpler: $d_i = T_current_i$ (plus boundary terms).
- void [solve_thomas_algorithm](#) ()
Solves the tridiagonal system using Thomas algorithm. Updates $T_solution_$ with the new temperature values.

Private Attributes

- double [r_](#)
CFL number.
- int [num_internal_nodes_](#)
Number of internal nodes (N-1).
- std::vector< double > [a_](#)
Sub-diagonal elements.
- std::vector< double > [b_](#)
Main diagonal elements.
- std::vector< double > [c_](#)
Super-diagonal elements.
- std::vector< double > [d_](#)
Right-hand side vector.
- std::vector< double > [T_solution_](#)
Temporary storage for solution.

5.3.1 Detailed Description

Implements the Laasonen implicit numerical scheme.

The Laasonen scheme is a fully implicit method (backward in time, central in space). It is unconditionally stable and first-order accurate in time, second-order in space. It requires solving a tridiagonal system of linear equations at each time step.

Definition at line 28 of file [Laasonen_Scheme.h](#).

5.3.2 Constructor & Destructor Documentation

5.3.2.1 Laasonen_Scheme()

```
Laasonen_Scheme::Laasonen_Scheme (
    const ProblemDefinition & problem)
```

Constructor.

Parameters

<i>problem</i>	The problem definition containing physical parameters.
----------------	--

Definition at line 21 of file [Laasonen_Scheme.cpp](#).

References [a_](#), [b_](#), [c_](#), [d_](#), [Scheme::Get_D\(\)](#), [Scheme::Get_dt\(\)](#), [ProblemDefinition::Get_dx\(\)](#), [Scheme::Get_dx\(\)](#), [Scheme::Get_N\(\)](#), [ProblemDefinition::Get_thickness\(\)](#), [num_internal_nodes_](#), [r_](#), [Scheme::Scheme\(\)](#), [setup_thomas_coefficients\(\)](#), and [T_solution_](#).

Here is the call graph for this function:

5.3.3 Member Function Documentation

5.3.3.1 setup_rhs_vector()

```
void Laasonen_Scheme::setup_rhs_vector (
    const std::vector< double > & T_current,
    double T_sur) [private]
```

Computes the RHS vector 'd'. Laasonen RHS is simpler: $d_i = T_current_i$ (plus boundary terms).

Parameters

<i>T_current</i>	The temperature distribution at the current time step.
<i>T_sur</i>	The surface temperature (boundary condition).

Definition at line 62 of file [Laasonen_Scheme.cpp](#).

References [d_](#), [num_internal_nodes_](#), and [r_](#).

Referenced by [solve_step\(\)](#).

Here is the caller graph for this function:

5.3.3.2 setup_thomas_coefficients()

```
void Laasonen_Scheme::setup_thomas_coefficients () [private]
```

Sets up the constant coefficients of the tridiagonal matrix. Laasonen: $a = -r$, $b = 1+2r$, $c = -r$.

Definition at line 47 of file [Laasonen_Scheme.cpp](#).

References [a_](#), [b_](#), [c_](#), [num_internal_nodes_](#), and [r_](#).

Referenced by [Laasonen_Scheme\(\)](#).

Here is the caller graph for this function:

5.3.3.3 Solve()

```
vector< vector< double > > Laasonen_Scheme::Solve ()
```

Solves the heat equation using the Laasonen scheme.

Returns

A 2D vector containing the temperature distribution over time and space.

Definition at line 122 of file [Laasonen_Scheme.cpp](#).

References [Scheme::Get_dx\(\)](#), [Scheme::Get_N\(\)](#), and [solve_step\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.3.3.4 solve_step()

```
void Laasonen_Scheme::solve_step (
    std::vector< double > & T_current,
    double T_sur)
```

Performs a single time step of the Laasonen scheme.

Parameters

<i>T_current</i>	The current temperature distribution (input/output).
<i>T_sur</i>	The surface temperature (boundary condition).

Definition at line 110 of file [Laasonen_Scheme.cpp](#).

References [num_internal_nodes_](#), [setup_rhs_vector\(\)](#), [solve_thomas_algorithm\(\)](#), and [T_solution_](#).

Referenced by [Solve\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.3.3.5 solve_thomas_algorithm()

```
void Laasonen_Scheme::solve_thomas_algorithm () [private]
```

Solves the tridiagonal system using Thomas algorithm. Updates [T_solution_](#) with the new temperature values.

Definition at line 83 of file [Laasonen_Scheme.cpp](#).

References [a_](#), [b_](#), [c_](#), [d_](#), [num_internal_nodes_](#), and [T_solution_](#).

Referenced by [solve_step\(\)](#).

Here is the caller graph for this function:

5.3.4 Member Data Documentation

5.3.4.1 a_

```
std::vector<double> Laasonen_Scheme::a_ [private]
```

Sub-diagonal elements.

Definition at line 35 of file [Laasonen_Scheme.h](#).

Referenced by [Laasonen_Scheme\(\)](#), [setup_thomas_coefficients\(\)](#), and [solve_thomas_algorithm\(\)](#).

5.3.4.2 b_

```
std::vector<double> Laasonen_Scheme::b_ [private]
```

Main diagonal elements.

Definition at line 36 of file [Laasonen_Scheme.h](#).

Referenced by [Laasonen_Scheme\(\)](#), [setup_thomas_coefficients\(\)](#), and [solve_thomas_algorithm\(\)](#).

5.3.4.3 c_

```
std::vector<double> Laasonen_Scheme::c_ [private]
```

Super-diagonal elements.

Definition at line 37 of file [Laasonen_Scheme.h](#).

Referenced by [Laasonen_Scheme\(\)](#), [setup_thomas_coefficients\(\)](#), and [solve_thomas_algorithm\(\)](#).

5.3.4.4 d_

```
std::vector<double> Laasonen_Scheme::d_ [private]
```

Right-hand side vector.

Definition at line 40 of file [Laasonen_Scheme.h](#).

Referenced by [Laasonen_Scheme\(\)](#), [setup_rhs_vector\(\)](#), and [solve_thomas_algorithm\(\)](#).

5.3.4.5 num_internal_nodes_

```
int Laasonen_Scheme::num_internal_nodes_ [private]
```

Number of internal nodes (N-1).

Definition at line 32 of file [Laasonen_Scheme.h](#).

Referenced by [Laasonen_Scheme\(\)](#), [setup_rhs_vector\(\)](#), [setup_thomas_coefficients\(\)](#), [solve_step\(\)](#), and [solve_thomas_algorithm\(\)](#).

5.3.4.6 r_

```
double Laasonen_Scheme::r_ [private]
```

CFL number.

Definition at line 31 of file [Laasonen_Scheme.h](#).

Referenced by [Laasonen_Scheme\(\)](#), [setup_rhs_vector\(\)](#), and [setup_thomas_coefficients\(\)](#).

5.3.4.7 T_solution_

```
std::vector<double> Laasonen_Scheme::T_solution_ [private]
```

Temporary storage for solution.

Definition at line 43 of file [Laasonen_Scheme.h](#).

Referenced by [Laasonen_Scheme\(\)](#), [solve_step\(\)](#), and [solve_thomas_algorithm\(\)](#).

The documentation for this class was generated from the following files:

- [Laasonen_Scheme.h](#)
- [Laasonen_Scheme.cpp](#)

5.4 Output Class Reference

Handles data export and visualization.

```
#include <Output.h>
```

Collaboration diagram for Output:

Public Member Functions

- [Output](#) ()
Default constructor.
- [Output](#) (string [filename](#))
Parameterized constructor.
- string [Get_filename](#) () const
Get filename.
- void [Set_filename](#) (string [value_filename](#))
Set filename.
- void [Generate_File](#) (const vector< vector< double > > &solution, string methodName, double dx, double dt, double tMax, int N)
Generates a CSV file containing the simulation results.
- void [Generate_Diagram](#) (const vector< vector< double > > &solution, string methodName, double dx, double dt, double tMax, int N)
Generates a plot of the solution using Gnuplot.

Private Attributes

- string [filename](#)
Base filename for output.

5.4.1 Detailed Description

Handles data export and visualization.

This class provides methods to export simulation results to CSV files and generate plots using Gnuplot.

Definition at line 24 of file [Output.h](#).

5.4.2 Constructor & Destructor Documentation

5.4.2.1 [Output\(\)](#) [1/2]

```
Output::Output ()
```

Default constructor.

Definition at line 22 of file [Output.cpp](#).

References [filename](#).

5.4.2.2 [Output\(\)](#) [2/2]

```
Output::Output (
    string filename)
```

Parameterized constructor.

Parameters

<i>filename</i>	Base filename for output
-----------------	--------------------------

Definition at line 28 of file [Output.cpp](#).

References [filename](#).

5.4.3 Member Function Documentation

5.4.3.1 Generate_Diagram()

```
void Output::Generate_Diagram (
    const vector< vector< double > > & solution,
    string methodName,
    double dx,
    double dt,
    double tMax,
    int N)
```

Generates a plot of the solution using Gnuplot.

This method pipes commands to a Gnuplot process to visualize the temperature profiles.

Parameters

<i>solution</i>	The 2D vector containing the solution [space][time]
<i>methodName</i>	The name of the numerical method (used for plot title)
<i>dx</i>	Spatial grid spacing
<i>dt</i>	Time step size
<i>tMax</i>	Maximum simulation time
<i>N</i>	Fourier Number

Definition at line 99 of file [Output.cpp](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:

5.4.3.2 Generate_File()

```
void Output::Generate_File (
    const vector< vector< double > > & solution,
    string methodName,
    double dx,
    double dt,
    double tMax,
    int N)
```

Generates a CSV file containing the simulation results.

The file format is compatible with Gnuplot and other plotting tools. It includes columns for position and temperature at selected time steps.

Parameters

<i>solution</i>	The 2D vector containing the solution [space][time]
<i>methodName</i>	The name of the numerical method (used for filename)
<i>dx</i>	Spatial grid spacing
<i>dt</i>	Time step size
<i>tMax</i>	Maximum simulation time
<i>N</i>	Fourier Number

Definition at line 46 of file [Output.cpp](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:

5.4.3.3 Get_filename()

```
string Output::Get_filename () const
```

Get filename.

Definition at line 34 of file [Output.cpp](#).

References [filename](#).

5.4.3.4 Set_filename()

```
void Output::Set_filename (
    string value_filename)
```

Set filename.

Definition at line 40 of file [Output.cpp](#).

References [filename](#).

5.4.4 Member Data Documentation

5.4.4.1 filename

```
string Output::filename [private]
```

Base filename for output.

Definition at line 27 of file [Output.h](#).

Referenced by [Get_filename\(\)](#), [Output\(\)](#), [Output\(\)](#), and [Set_filename\(\)](#).

The documentation for this class was generated from the following files:

- [Output.h](#)
- [Output.cpp](#)

5.5 ProblemDefinition Class Reference

```
#include <ProblemDefinition.h>
```

Collaboration diagram for ProblemDefinition:

Public Member Functions

- [ProblemDefinition](#) ()
Default constructor. Initializes all parameters to zero.
- [ProblemDefinition](#) (double [thickness](#), double [dx](#), double [tMax](#), double [dt](#), int [N](#), double [Tin](#), double [Tsur](#), double [D](#))
Parameterized constructor.
- double [Get_dx](#) () const
Gets the spatial grid spacing.
- double [Get_tMax](#) () const
Gets the maximum simulation time.
- double [Get_dt](#) () const
Gets the time step size.
- int [Get_N](#) () const
Gets the number of grid points.
- double [Get_Tin](#) () const
Gets the initial temperature.
- double [Get_Tsur](#) () const
Gets the surface temperature.
- double [Get_thickness](#) () const
Gets the material thickness.
- double [Get_D](#) () const
Gets the thermal diffusivity.
- void [Set_dx](#) (double value)
Sets the spatial grid spacing.
- void [Set_tMax](#) (double value)
Sets the maximum simulation time.
- void [Set_dt](#) (double value)
Sets the time step size.
- void [Set_N](#) (int value)
Sets the number of grid points.
- void [Set_Tin](#) (double value)
Sets the initial temperature.
- void [Set_Tsur](#) (double value)
Sets the surface temperature.
- void [Set_thickness](#) (double value)
Sets the material thickness.
- void [Set_D](#) (double value)
Sets the thermal diffusivity.
- vector< vector< double > > [Analytic_Solution](#) ()
Computes the analytic solution for the heat equation.

Private Attributes

- double `dx` = 0.0
Spatial grid spacing [m].
- double `tMax` = 0.0
Maximum simulation time [s].
- double `dt` = 0.0
Time step size [s].
- int `N` = 0
Fourier Number.
- double `Tin` = 0.0
Initial temperature [K].
- double `Tsur` = 0.0
Surface temperature [K].
- double `thickness` = 0.0
Material thickness [m].
- double `D` = 0.0
Thermal diffusivity [m^2/s].

5.5.1 Detailed Description

Definition at line 17 of file [ProblemDefinition.h](#).

5.5.2 Constructor & Destructor Documentation

5.5.2.1 ProblemDefinition() [1/2]

```
ProblemDefinition::ProblemDefinition ()
```

Default constructor. Initializes all parameters to zero.

Definition at line 21 of file [ProblemDefinition.cpp](#).

References [D](#), [dt](#), [dx](#), [N](#), [thickness](#), [Tin](#), [tMax](#), and [Tsur](#).

5.5.2.2 ProblemDefinition() [2/2]

```
ProblemDefinition::ProblemDefinition (
    double thickness,
    double dx,
    double tMax,
    double dt,
    int N,
    double Tin,
    double Tsur,
    double D)
```

Parameterized constructor.

Parameters

<i>thickness</i>	Material thickness [m]
<i>dx</i>	Spatial grid spacing [m]
<i>tMax</i>	Maximum simulation time [s]
<i>dt</i>	Time step size [s]
<i>N</i>	Fourier Number
<i>Tin</i>	Initial temperature [K]
<i>Tsur</i>	Surface temperature [K]
<i>D</i>	Thermal diffusivity [m^2/s]

Definition at line 40 of file [ProblemDefinition.cpp](#).

References [D](#), [dt](#), [dx](#), [N](#), [thickness](#), [Tin](#), [tMax](#), and [Tsur](#).

5.5.3 Member Function Documentation

5.5.3.1 Analytic_Solution()

```
vector< vector< double > > ProblemDefinition::Analytic_Solution ()
```

Computes the analytic solution for the heat equation.

Returns

A 2D vector representing the temperature distribution over time and space.

Referenced by [main\(\)](#).

Here is the caller graph for this function:

5.5.3.2 Get_D()

```
double ProblemDefinition::Get_D () const
```

Gets the thermal diffusivity.

Returns

D [m^2/s]

Definition at line 94 of file [ProblemDefinition.cpp](#).

References [D](#).

5.5.3.3 Get_dt()

```
double ProblemDefinition::Get_dt () const
```

Gets the time step size.

Returns

dt [s]

Definition at line 69 of file [ProblemDefinition.cpp](#).

References [dt](#).

5.5.3.4 Get_dx()

```
double ProblemDefinition::Get_dx () const
```

Gets the spatial grid spacing.

Returns

dx [m]

Definition at line 59 of file [ProblemDefinition.cpp](#).

References [dx](#).

Referenced by [CrankNicholson_Scheme::CrankNicholson_Scheme\(\)](#), and [Laasonen_Scheme::Laasonen_Scheme\(\)](#).

Here is the caller graph for this function:

5.5.3.5 Get_N()

```
int ProblemDefinition::Get_N () const
```

Gets the number of grid points.

Returns

N (Fourier Number)

Definition at line 74 of file [ProblemDefinition.cpp](#).

References [N](#).

5.5.3.6 Get_thickness()

```
double ProblemDefinition::Get_thickness () const
```

Gets the material thickness.

Returns

thickness [m]

Definition at line 89 of file [ProblemDefinition.cpp](#).

References [thickness](#).

Referenced by [CrankNicholson_Scheme::CrankNicholson_Scheme\(\)](#), and [Laasonen_Scheme::Laasonen_Scheme\(\)](#).

Here is the caller graph for this function:

5.5.3.7 Get_Tin()

```
double ProblemDefinition::Get_Tin () const
```

Gets the initial temperature.

Returns

Tin [K]

Definition at line 79 of file [ProblemDefinition.cpp](#).

References [Tin](#).

5.5.3.8 Get_tMax()

```
double ProblemDefinition::Get_tMax () const
```

Gets the maximum simulation time.

Returns

tMax [s]

Definition at line 64 of file [ProblemDefinition.cpp](#).

References [tMax](#).

5.5.3.9 Get_Tsur()

```
double ProblemDefinition::Get_Tsur () const
```

Gets the surface temperature.

Returns

Tsur [K]

Definition at line 84 of file [ProblemDefinition.cpp](#).

References [Tsur](#).

5.5.3.10 Set_D()

```
void ProblemDefinition::Set_D (  
    double value)
```

Sets the thermal diffusivity.

Parameters

<i>value</i>	D [m ² /s]
--------------	-----------------------

Definition at line 135 of file [ProblemDefinition.cpp](#).

References [D](#).

5.5.3.11 Set_dt()

```
void ProblemDefinition::Set_dt (  
    double value)
```

Sets the time step size.

Parameters

<i>value</i>	dt [s]
--------------	--------

Definition at line 110 of file [ProblemDefinition.cpp](#).

References [dt](#).

5.5.3.12 Set_dx()

```
void ProblemDefinition::Set_dx (  
    double value)
```

Sets the spatial grid spacing.

Parameters

<i>value</i>	dx [m]
--------------	--------

Definition at line 100 of file [ProblemDefinition.cpp](#).

References [dx](#).

5.5.3.13 Set_N()

```
void ProblemDefinition::Set_N (  
    int value)
```

Sets the number of grid points.

Parameters

<i>value</i>	N (Fourier Number)
--------------	--------------------

Definition at line 115 of file [ProblemDefinition.cpp](#).

References [N](#).

5.5.3.14 Set_thickness()

```
void ProblemDefinition::Set_thickness (
    double value)
```

Sets the material thickness.

Parameters

<i>value</i>	thickness [m]
--------------	---------------

Definition at line 130 of file [ProblemDefinition.cpp](#).

References [thickness](#).

5.5.3.15 Set_Tin()

```
void ProblemDefinition::Set_Tin (
    double value)
```

Sets the initial temperature.

Parameters

<i>value</i>	Tin [K]
--------------	---------

Definition at line 120 of file [ProblemDefinition.cpp](#).

References [Tin](#).

5.5.3.16 Set_tMax()

```
void ProblemDefinition::Set_tMax (
    double value)
```

Sets the maximum simulation time.

Parameters

<i>value</i>	tMax [s]
--------------	----------

Definition at line 105 of file [ProblemDefinition.cpp](#).

References [tMax](#).

5.5.3.17 Set_Tsur()

```
void ProblemDefinition::Set_Tsur (  
    double value)
```

Sets the surface temperature.

Parameters

<i>value</i>	Tsur [K]
--------------	----------

Definition at line 125 of file [ProblemDefinition.cpp](#).

References [Tsur](#).

5.5.4 Member Data Documentation

5.5.4.1 D

```
double ProblemDefinition::D = 0.0 [private]
```

Thermal diffusivity [m^2/s].

Definition at line 28 of file [ProblemDefinition.h](#).

Referenced by [Get_D\(\)](#), [ProblemDefinition\(\)](#), [ProblemDefinition\(\)](#), and [Set_D\(\)](#).

5.5.4.2 dt

```
double ProblemDefinition::dt = 0.0 [private]
```

Time step size [s].

Definition at line 23 of file [ProblemDefinition.h](#).

Referenced by [Get_dt\(\)](#), [ProblemDefinition\(\)](#), [ProblemDefinition\(\)](#), and [Set_dt\(\)](#).

5.5.4.3 dx

```
double ProblemDefinition::dx = 0.0 [private]
```

Spatial grid spacing [m].

Definition at line 21 of file [ProblemDefinition.h](#).

Referenced by [Get_dx\(\)](#), [ProblemDefinition\(\)](#), [ProblemDefinition\(\)](#), and [Set_dx\(\)](#).

5.5.4.4 N

```
int ProblemDefinition::N = 0 [private]
```

Fourier Number.

Definition at line 24 of file [ProblemDefinition.h](#).

Referenced by [Get_N\(\)](#), [ProblemDefinition\(\)](#), [ProblemDefinition\(\)](#), and [Set_N\(\)](#).

5.5.4.5 thickness

```
double ProblemDefinition::thickness = 0.0 [private]
```

Material thickness [m].

Definition at line 27 of file [ProblemDefinition.h](#).

Referenced by [Get_thickness\(\)](#), [ProblemDefinition\(\)](#), [ProblemDefinition\(\)](#), and [Set_thickness\(\)](#).

5.5.4.6 Tin

```
double ProblemDefinition::Tin = 0.0 [private]
```

Initial temperature [K].

Definition at line 25 of file [ProblemDefinition.h](#).

Referenced by [Get_Tin\(\)](#), [ProblemDefinition\(\)](#), [ProblemDefinition\(\)](#), and [Set_Tin\(\)](#).

5.5.4.7 tMax

```
double ProblemDefinition::tMax = 0.0 [private]
```

Maximum simulation time [s].

Definition at line 22 of file [ProblemDefinition.h](#).

Referenced by [Get_tMax\(\)](#), [ProblemDefinition\(\)](#), [ProblemDefinition\(\)](#), and [Set_tMax\(\)](#).

5.5.4.8 Tsur

```
double ProblemDefinition::Tsur = 0.0 [private]
```

Surface temperature [K].

Definition at line 26 of file [ProblemDefinition.h](#).

Referenced by [Get_Tsur\(\)](#), [ProblemDefinition\(\)](#), [ProblemDefinition\(\)](#), and [Set_Tsur\(\)](#).

The documentation for this class was generated from the following files:

- [ProblemDefinition.h](#)
- [ProblemDefinition.cpp](#)

5.6 Richardson_Scheme Class Reference

Implements the Richardson explicit numerical scheme.

```
#include <Richardson_Scheme.h>
```

Inheritance diagram for Richardson_Scheme:

Collaboration diagram for Richardson_Scheme:

Public Member Functions

- [Richardson_Scheme](#) (const [ProblemDefinition](#) &problem)
Constructor.
- `vector< vector< double > > richardsonSolution` (vector< vector< double > > laasonen)
Solves the heat equation using the Richardson scheme.

Public Member Functions inherited from [Scheme](#)

- [Scheme](#) ()
Default constructor.
- [Scheme](#) (double dx, int N, double dt, double D)
Parameterized constructor.
- double [Get_dx](#) () const
Get spatial grid spacing.
- int [Get_N](#) () const
Get Fourier Number.
- double [Get_dt](#) () const
Get time step size.
- double [Get_D](#) () const
Get thermal diffusivity.
- void [Set_dx](#) (double value)
Set spatial grid spacing.
- void [Set_N](#) (int value)
Set Fourier Number.
- void [Set_dt](#) (double value)
Set time step size.
- void [Set_D](#) (double value)
Set thermal diffusivity.
- double [Get_CFL](#) ()
Calculates and returns the CFL number.

Private Attributes

- double r = 0.0
CFL number.
- [ProblemDefinition](#) problem
Problem definition object.

5.6.1 Detailed Description

Implements the Richardson explicit numerical scheme.

The Richardson scheme is a second-order accurate explicit method in both time and space. However, it is unconditionally unstable for the 1D diffusion equation.

Definition at line 28 of file [Richardson_Scheme.h](#).

5.6.2 Constructor & Destructor Documentation

5.6.2.1 Richardson_Scheme()

```
Richardson_Scheme::Richardson_Scheme (  
    const ProblemDefinition & problem)
```

Constructor.

Parameters

<i>problem</i>	The problem definition containing physical parameters.
----------------	--

Definition at line 20 of file [Richardson_Scheme.cpp](#).

References [Scheme::Get_CFL\(\)](#), [Scheme::Get_D\(\)](#), [Scheme::Get_dt\(\)](#), [Scheme::Get_dx\(\)](#), [Scheme::Get_N\(\)](#), [problem](#), [r](#), and [Scheme::Scheme\(\)](#).

Here is the call graph for this function:

5.6.3 Member Function Documentation

5.6.3.1 richardsonSolution()

```
vector< vector< double > > Richardson_Scheme::richardsonSolution (
    vector< vector< double > > laasonen)
```

Solves the heat equation using the Richardson scheme.

Parameters

<i>laasonen</i>	The solution from the Laasonen scheme (used for the first time step).
-----------------	---

Returns

A 2D vector containing the temperature distribution over time and space.

Definition at line 28 of file [Richardson_Scheme.cpp](#).

References [problem](#), and [r](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:

5.6.4 Member Data Documentation

5.6.4.1 problem

```
ProblemDefinition Richardson_Scheme::problem [private]
```

Problem definition object.

Definition at line 32 of file [Richardson_Scheme.h](#).

Referenced by [Richardson_Scheme\(\)](#), and [richardsonSolution\(\)](#).

5.6.4.2 r

```
double Richardson_Scheme::r = 0.0 [private]
```

CFL number.

Definition at line 31 of file [Richardson_Scheme.h](#).

Referenced by [Richardson_Scheme\(\)](#), and [richardsonSolution\(\)](#).

The documentation for this class was generated from the following files:

- [Richardson_Scheme.h](#)
- [Richardson_Scheme.cpp](#)

5.7 Scheme Class Reference

Base class for numerical schemes.

```
#include <Scheme.h>
```

Inheritance diagram for Scheme:

Collaboration diagram for Scheme:

Public Member Functions

- [Scheme](#) ()
Default constructor.
- [Scheme](#) (double [dx](#), int [N](#), double [dt](#), double [D](#))
Parameterized constructor.
- double [Get_dx](#) () const
Get spatial grid spacing.
- int [Get_N](#) () const
Get Fourier Number.
- double [Get_dt](#) () const
Get time step size.
- double [Get_D](#) () const
Get thermal diffusivity.
- void [Set_dx](#) (double value)
Set spatial grid spacing.
- void [Set_N](#) (int value)
Set Fourier Number.
- void [Set_dt](#) (double value)
Set time step size.
- void [Set_D](#) (double value)
Set thermal diffusivity.
- double [Get_CFL](#) ()
Calculates and returns the CFL number.

Private Attributes

- double `dx`
Spatial grid spacing (cm).
- int `N`
Fourier Number.
- double `dt`
Time step size (h).
- double `D`
Thermal diffusivity (cm²/h).
- double `r`
CFL number (Courant-Friedrichs-Lewy condition).

5.7.1 Detailed Description

Base class for numerical schemes.

This class serves as a parent class for all specific numerical schemes (e.g., Dufort-Frankel, Richardson, Laasonen, Crank-Nicholson). It holds common parameters like grid spacing, time step, and material properties.

Definition at line 23 of file [Scheme.h](#).

5.7.2 Constructor & Destructor Documentation

5.7.2.1 `Scheme()` [1/2]

```
Scheme::Scheme ()
```

Default constructor.

Definition at line 17 of file [Scheme.cpp](#).

References [D](#), [dt](#), [dx](#), [N](#), and [r](#).

Referenced by [CrankNicholson_Scheme::CrankNicholson_Scheme\(\)](#), [DufortFrankel_Scheme::DufortFrankel_Scheme\(\)](#), [Laasonen_Scheme::Laasonen_Scheme\(\)](#), and [Richardson_Scheme::Richardson_Scheme\(\)](#).

Here is the caller graph for this function:

5.7.2.2 `Scheme()` [2/2]

```
Scheme::Scheme (
    double dx,
    int N,
    double dt,
    double D)
```

Parameterized constructor.

Parameters

dx	Spatial grid spacing (cm)
N	Fourier Number
dt	Time step size (h)
D	Thermal diffusivity (cm ² /h)

Definition at line 27 of file [Scheme.cpp](#).

References [D](#), [dt](#), [dx](#), and [N](#).

5.7.3 Member Function Documentation

5.7.3.1 Get_CFL()

```
double Scheme::Get_CFL ()
```

Calculates and returns the CFL number.

Returns

The CFL number ($r = D * dt / dx^2$).

Definition at line 79 of file [Scheme.cpp](#).

References [D](#), [dt](#), [dx](#), and [r](#).

Referenced by [DufortFrankel_Scheme::DufortFrankel_Scheme\(\)](#), and [Richardson_Scheme::Richardson_Scheme\(\)](#).

Here is the caller graph for this function:

5.7.3.2 Get_D()

```
double Scheme::Get_D () const
```

Get thermal diffusivity.

Definition at line 51 of file [Scheme.cpp](#).

References [D](#).

Referenced by [CrankNicholson_Scheme::CrankNicholson_Scheme\(\)](#), [DufortFrankel_Scheme::DufortFrankel_Scheme\(\)](#), [Laasonen_Scheme::Laasonen_Scheme\(\)](#), and [Richardson_Scheme::Richardson_Scheme\(\)](#).

Here is the caller graph for this function:

5.7.3.3 Get_dt()

```
double Scheme::Get_dt () const
```

Get time step size.

Definition at line 46 of file [Scheme.cpp](#).

References [dt](#).

Referenced by [CrankNicholson_Scheme::CrankNicholson_Scheme\(\)](#), [DufortFrankel_Scheme::DufortFrankel_Scheme\(\)](#), [Laasonen_Scheme::Laasonen_Scheme\(\)](#), and [Richardson_Scheme::Richardson_Scheme\(\)](#).

Here is the caller graph for this function:

5.7.3.4 Get_dx()

```
double Scheme::Get_dx () const
```

Get spatial grid spacing.

Definition at line 36 of file [Scheme.cpp](#).

References [dx](#).

Referenced by [CrankNicholson_Scheme::CrankNicholson_Scheme\(\)](#), [DufortFrankel_Scheme::DufortFrankel_Scheme\(\)](#), [Laasonen_Scheme::Laasonen_Scheme\(\)](#), [Richardson_Scheme::Richardson_Scheme\(\)](#), [CrankNicholson_Scheme::Solve\(\)](#), and [Laasonen_Scheme::Solve\(\)](#).

Here is the caller graph for this function:

5.7.3.5 Get_N()

```
int Scheme::Get_N () const
```

Get Fourier Number.

Definition at line 41 of file [Scheme.cpp](#).

References [N](#).

Referenced by [CrankNicholson_Scheme::CrankNicholson_Scheme\(\)](#), [DufortFrankel_Scheme::DufortFrankel_Scheme\(\)](#), [Laasonen_Scheme::Laasonen_Scheme\(\)](#), [Richardson_Scheme::Richardson_Scheme\(\)](#), [CrankNicholson_Scheme::Solve\(\)](#), and [Laasonen_Scheme::Solve\(\)](#).

Here is the caller graph for this function:

5.7.3.6 Set_D()

```
void Scheme::Set_D (  
    double value)
```

Set thermal diffusivity.

Definition at line 72 of file [Scheme.cpp](#).

References [D](#).

5.7.3.7 Set_dt()

```
void Scheme::Set_dt (
    double value)
```

Set time step size.

Definition at line 67 of file [Scheme.cpp](#).

References [dt](#).

5.7.3.8 Set_dx()

```
void Scheme::Set_dx (
    double value)
```

Set spatial grid spacing.

Definition at line 57 of file [Scheme.cpp](#).

References [dx](#).

5.7.3.9 Set_N()

```
void Scheme::Set_N (
    int value)
```

Set Fourier Number.

Definition at line 62 of file [Scheme.cpp](#).

References [N](#).

5.7.4 Member Data Documentation

5.7.4.1 D

```
double Scheme::D [private]
```

Thermal diffusivity (cm²/h).

Definition at line 29 of file [Scheme.h](#).

Referenced by [Get_CFL\(\)](#), [Get_D\(\)](#), [Scheme\(\)](#), [Scheme\(\)](#), and [Set_D\(\)](#).

5.7.4.2 dt

```
double Scheme::dt [private]
```

Time step size (h).

Definition at line 28 of file [Scheme.h](#).

Referenced by [Get_CFL\(\)](#), [Get_dt\(\)](#), [Scheme\(\)](#), [Scheme\(\)](#), and [Set_dt\(\)](#).

5.7.4.3 dx

```
double Scheme::dx [private]
```

Spatial grid spacing (cm).

Definition at line 26 of file [Scheme.h](#).

Referenced by [Get_CFL\(\)](#), [Get_dx\(\)](#), [Scheme\(\)](#), [Scheme\(\)](#), and [Set_dx\(\)](#).

5.7.4.4 N

```
int Scheme::N [private]
```

Fourier Number.

Definition at line 27 of file [Scheme.h](#).

Referenced by [Get_N\(\)](#), [Scheme\(\)](#), [Scheme\(\)](#), and [Set_N\(\)](#).

5.7.4.5 r

```
double Scheme::r [private]
```

CFL number (Courant-Friedrichs-Lewy condition).

Definition at line 30 of file [Scheme.h](#).

Referenced by [Get_CFL\(\)](#), and [Scheme\(\)](#).

The documentation for this class was generated from the following files:

- [Scheme.h](#)
- [Scheme.cpp](#)

5.8 Verification Class Reference

Provides methods for verifying numerical results and conditions.

```
#include <Verification.h>
```

Collaboration diagram for Verification:

Public Member Functions

- [Verification](#) ()
Default constructor.
- [Verification](#) (double [CFL](#))
Parameterized constructor.
- double [Get_CFL](#) () const
Get CFL number.
- void [Set_CFL](#) (double value)
Set CFL number.
- bool [Verify_Stability](#) (double error, double tolerance)
Verifies the stability of a numerical scheme based on error and tolerance.
- double [Calculation_L2_Norm](#) (const vector< double > &numerical, const vector< double > &analytical, double dx)
Calculates the L2 Norm (Discrete Integral) between numerical and analytical solutions.
- double [Calculation_Relative_Error](#) (const vector< double > &numerical, const vector< double > &analytical, double dx)
Calculates the Relative L2 Error.

Private Attributes

- double [CFL](#)
Courant-Friedrichs-Lewy number.

5.8.1 Detailed Description

Provides methods for verifying numerical results and conditions.

This class includes utilities to check the stability of numerical schemes and calculate error norms (e.g., L2 norm) by comparing numerical results with analytical solutions.

Definition at line 26 of file [Verification.h](#).

5.8.2 Constructor & Destructor Documentation**5.8.2.1 Verification() [1/2]**

```
Verification::Verification ()
```

Default constructor.

Definition at line 21 of file [Verification.cpp](#).

References [CFL](#).

5.8.2.2 Verification() [2/2]

```
Verification::Verification (
    double CFL)
```

Parameterized constructor.

Parameters

<i>CFL</i>	The CFL number to check.
------------	--------------------------

Definition at line 27 of file [Verification.cpp](#).

References [CFL](#).

5.8.3 Member Function Documentation

5.8.3.1 Calculation_L2_Norm()

```
double Verification::Calculation_L2_Norm (
    const vector< double > & numerical,
    const vector< double > & analytical,
    double dx)
```

Calculates the L2 Norm (Discrete Integral) between numerical and analytical solutions.

Formula: $\sqrt{\sum (num[i] - ana[i])^2 * dx}$

Parameters

<i>numerical</i>	Vector containing the numerical solution.
<i>analytical</i>	Vector containing the analytical solution.
<i>dx</i>	The spatial grid spacing.

Returns

The calculated L2 Norm. Returns -1.0 if vector sizes do not match.

Definition at line 62 of file [Verification.cpp](#).

Referenced by [Calculation_Relative_Error\(\)](#), and [main\(\)](#).

Here is the caller graph for this function:

5.8.3.2 Calculation_Relative_Error()

```
double Verification::Calculation_Relative_Error (
    const vector< double > & numerical,
    const vector< double > & analytical,
    double dx)
```

Calculates the Relative L2 Error.

Formula: $L2_Norm(num - ana) / L2_Norm(ana)$

Parameters

<i>numerical</i>	Vector containing the numerical solution.
<i>analytical</i>	Vector containing the analytical solution.
<i>dx</i>	The spatial grid spacing.

Returns

The calculated Relative Error. Returns -1.0 if vector sizes do not match or analytical norm is 0.

Definition at line 81 of file [Verification.cpp](#).

References [Calculation_L2_Norm\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function: Here is the caller graph for this function:

5.8.3.3 Get_CFL()

```
double Verification::Get_CFL () const
```

Get CFL number.

Definition at line 33 of file [Verification.cpp](#).

References [CFL](#).

5.8.3.4 Set_CFL()

```
void Verification::Set_CFL (
    double value)
```

Set CFL number.

Definition at line 39 of file [Verification.cpp](#).

References [CFL](#).

5.8.3.5 Verify_Stability()

```
bool Verification::Verify_Stability (
    double error,
    double tolerance)
```

Verifies the stability of a numerical scheme based on error and tolerance.

Parameters

<i>error</i>	The calculated error (e.g., L2 Norm).
<i>tolerance</i>	The acceptable error threshold.

Returns

true if stable (error $\leq$ tolerance and not NaN/Inf), false otherwise.

Definition at line 46 of file [Verification.cpp](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:

5.8.4 Member Data Documentation

5.8.4.1 CFL

```
double Verification::CFL [private]
```

Courant-Friedrichs-Lewy number.

Definition at line 29 of file [Verification.h](#).

Referenced by [Get_CFL\(\)](#), [Set_CFL\(\)](#), [Verification\(\)](#), and [Verification\(\)](#).

The documentation for this class was generated from the following files:

- [Verification.h](#)
- [Verification.cpp](#)

Chapter 6

File Documentation

6.1 CrankNicholson_Scheme.cpp File Reference

```
#include "CrankNicholson_Scheme.h"
#include "ProblemDefinition.h"
#include <iostream>
#include <vector>
#include <cmath>
Include dependency graph for CrankNicholson_Scheme.cpp:
```

6.2 CrankNicholson_Scheme.cpp

[Go to the documentation of this file.](#)

```
00001 /* Computational Methods Assignment
00002    @author :      John Hoarau, Github copilot
00003    @date :      06/11/2025
00004    @file :      CrankNicholson_Scheme.cpp
00005    @description: Implementation of the CrankNicholson_Scheme class.
00006 */
00007
00008 /***** Inclusion of classes *****/
00009 #include "CrankNicholson_Scheme.h"
00010 #include "ProblemDefinition.h"
00011
00012 /***** Libraries and other inclusions *****/
00013 #include <iostream>
00014 #include <vector>
00015 #include <cmath>
00016
00017 using namespace std;
00018
00019 /***** Methods for class CrankNicholson_Scheme *****/
00020
00021 CrankNicholson_Scheme::CrankNicholson_Scheme(const ProblemDefinition& problem)
00022     // Call the base class 'Scheme' constructor
00023     : Scheme(problem.Get_dx(), problem.Get_N(), problem.Get_dt(), problem.Get_D())
00024 {
00025     // Calculate the 'r' parameter
00026     this->r_ = (Get_D() * Get_dt()) / (Get_dx() * Get_dx());
00027
00028     // FIX: Calculate spatial nodes based on geometry, NOT time steps (N)
00029     int num_space_points = static_cast<int>(problem.Get_thickness() / problem.Get_dx()) + 1;
00030
00031     // The tridiagonal system only applies to INTERNAL nodes
00032     this->num_internal_nodes_ = num_space_points - 2;
00033
00034     // Resize vectors to fit the internal nodes
00035     if (num_internal_nodes_ > 0) {
00036         a_.resize(num_internal_nodes_);
00037         b_.resize(num_internal_nodes_);
```

```

00038         c_.resize(num_internal_nodes_);
00039         d_.resize(num_internal_nodes_);
00040         T_solution_.resize(num_internal_nodes_);
00041
00042         // Set up the (constant) matrix A coefficients
00043         setup_thomas_coefficients();
00044     } else {
00045         // Handle the case where there are no internal nodes (e.g., N=2)
00046         std::cerr << "ERROR: Number of spatial nodes must be > 2." << std::endl;
00047     }
00048 }
00049
00050 void CrankNicholson_Scheme::setup_thomas_coefficients() {
00051     double val_a = -r_ / 2.0;
00052     double val_b = 1.0 + r_;
00053     double val_c = -r_ / 2.0;
00054
00055     for (int i = 0; i < num_internal_nodes_; ++i) {
00056         a_[i] = val_a;
00057         b_[i] = val_b;
00058         c_[i] = val_c;
00059     }
00060 }
00061
00062 // Sets up the RHS vector 'd_' based on the current temperature distribution and boundary conditions
00063 void CrankNicholson_Scheme::setup_rhs_vector(const std::vector<double>& T_current, double T_sur) {
00064
00065     // 1. Middle internal nodes (from i_int = 1 to N-4)
00066     // (corresponding to physical nodes j=2 to N-3)
00067     for (int i_int = 1; i_int < num_internal_nodes_ - 1; ++i_int) {
00068         int j_phys = i_int + 1; // Corresponding physical node
00069         d_[i_int] = (r_ / 2.0) * T_current[j_phys - 1] +
00070                     (1.0 - r_) * T_current[j_phys] +
00071                     (r_ / 2.0) * T_current[j_phys + 1];
00072     }
00073
00074     // 2. Handle the first internal node (i_int = 0, j_phys = 1)
00075     if (num_internal_nodes_ > 0) {
00076         d_[0] = r_ * T_sur +
00077                 (1.0 - r_) * T_current[1] +
00078                 (r_ / 2.0) * T_current[2];
00079     }
00080
00081     // 3. Handle the last internal node (i_int = N-3, j_phys = N-2)
00082     if (num_internal_nodes_ > 1) {
00083         int i_last = num_internal_nodes_ - 1;
00084         int j_last = i_last + 1;
00085
00086         d_[i_last] = (r_ / 2.0) * T_current[j_last - 1] +
00087                     (1.0 - r_) * T_current[j_last] +
00088                     r_ * T_sur;
00089     }
00090 }
00091
00092
00093 // Solves the tridiagonal system using the Thomas algorithm
00094 void CrankNicholson_Scheme::solve_thomas_algorithm() {
00095     if (num_internal_nodes_ <= 0) return;
00096
00097     // Create temporary copies for the elimination pass
00098     vector<double> c_prime = c_;
00099     vector<double> d_prime = d_;
00100
00101     // 1. Forward Elimination pass
00102     c_prime[0] = c_prime[0] / b_[0];
00103     d_prime[0] = d_prime[0] / b_[0];
00104
00105     for (int i = 1; i < num_internal_nodes_; ++i) {
00106         double m = b_[i] - a_[i] * c_prime[i - 1];
00107         if (std::abs(m) < 1e-10) {
00108             cerr << "ERROR: Thomas algorithm unstable (division by zero)." << endl;
00109             return;
00110         }
00111         c_prime[i] = c_prime[i] / m;
00112         d_prime[i] = (d_prime[i] - a_[i] * d_prime[i - 1]) / m;
00113     }
00114
00115     // 2. Backward Substitution pass
00116     T_solution_[num_internal_nodes_ - 1] = d_prime[num_internal_nodes_ - 1];
00117
00118     for (int i = num_internal_nodes_ - 2; i >= 0; --i) {
00119         T_solution_[i] = d_prime[i] - c_prime[i] * T_solution_[i + 1];
00120     }
00121 }
00122
00123 // Performs one time step of the Crank-Nicholson scheme
00124 void CrankNicholson_Scheme::solve_step(std::vector<double>& T_current, double T_sur) {

```

```

00125     if (num_internal_nodes_ <= 0) return;
00126
00127     setup_rhs_vector(T_current, T_sur);
00128     solve_thomas_algorithm();
00129
00130     for (int i = 0; i < num_internal_nodes_; ++i) {
00131         T_current[i + 1] = T_solution_[i];
00132     }
00133 }
00134
00135 vector<vector<double>> CrankNicholson_Scheme::Solve() {
00136     int num_time_steps = static_cast<int>(Get_N()) + 1;
00137     // Calculate spatial points based on thickness and dx
00138     int num_space_points = static_cast<int>(31.0 / Get_dx()) + 1;
00139
00140     // Initialize solution matrix [space x time]
00141     vector<vector<double>> solution(num_space_points, vector<double>(num_time_steps, 0.0));
00142
00143     // Current temperature profile
00144     vector<double> T_curr(num_space_points);
00145
00146     // 1. Set Initial Conditions (t=0)
00147     for (int i = 0; i < num_space_points; ++i) {
00148         T_curr[i] = 38.0; // Tin
00149     }
00150
00151     // Boundary Conditions
00152     T_curr[0] = 149.0; // Tsur
00153     T_curr[num_space_points - 1] = 149.0;
00154
00155     // Store IC into solution column 0
00156     for (int i = 0; i < num_space_points; ++i) {
00157         solution[i][0] = T_curr[i];
00158     }
00159
00160     // 2. Time Loop
00161     for (int t = 1; t < num_time_steps; ++t) {
00162         solve_step(T_curr, 149.0);
00163
00164         for (int i = 0; i < num_space_points; ++i) {
00165             solution[i][t] = T_curr[i];
00166         }
00167     }
00168
00169     return solution;
00170 }

```

6.3 CrankNicholson_Scheme.h File Reference

```

#include "Scheme.h"
#include "ProblemDefinition.h"
#include <vector>
#include <iostream>

```

Include dependency graph for CrankNicholson_Scheme.h: This graph shows which files directly or indirectly include this file:

Classes

- class [CrankNicholson_Scheme](#)
Implements the Crank-Nicholson implicit numerical scheme.

6.4 CrankNicholson_Scheme.h

[Go to the documentation of this file.](#)

```

00001 /* Computational Methods Assignment
00002    @author :      John Hoarau, Github copilot
00003    @date :      16/10/2025

```

```

00004      @file :          CrankNicholson_Scheme.h
00005      @description:    Header for the Crank-Nicholson implicit scheme.
00006  */
00007
00008  /***** Prevention for multiple definition *****/
00009  #ifndef CRANKNICHOLSON_SCHEME_H
00010  #define CRANKNICHOLSON_SCHEME_H
00011
00012  /***** Libraries and other inclusions *****/
00013  #include "Scheme.h"
00014  #include "ProblemDefinition.h" // Nécessaire pour le constructeur
00015  #include <vector>
00016  #include <iostream>
00017
00018  using namespace std;
00019
00020  /***** Declaration of class CrankNicholson_Scheme *****/
00021
00030  class CrankNicholson_Scheme : public Scheme
00031  {
00032      private:
00033          double r_;
00034          int num_internal_nodes_;
00035
00036          // Coefficients for the Tridiagonal Matrix System ( $A * T^{n+1} = RHS$ )
00037          std::vector<double> a_;
00038          std::vector<double> b_;
00039          std::vector<double> c_;
00040
00041          std::vector<double> d_;
00042
00043          std::vector<double> T_solution_;
00044
00049          void setup_thomas_coefficients();
00050
00057          void setup_rhs_vector(const std::vector<double>& T_current, double T_sur);
00058
00063          void solve_thomas_algorithm();
00064
00065      public:
00070          CrankNicholson_Scheme(const ProblemDefinition& problem);
00071
00076          vector<vector<double>> Solve();
00077
00083          void solve_step(std::vector<double>& T_current, double T_sur);
00084
00085  };
00086
00087  #endif // CRANKNICHOLSON_SCHEME_H

```

6.5 DufortFrankel_Scheme.cpp File Reference

```

#include "DufortFrankel_Scheme.h"
#include "ProblemDefinition.h"
#include <cmath>
#include <vector>

```

Include dependency graph for DufortFrankel_Scheme.cpp:

6.6 DufortFrankel_Scheme.cpp

[Go to the documentation of this file.](#)

```

00001  /* Computational Methods Assignment
00002  @author :          Clémence-Philomène Hinot
00003  @date :            27/11/2025
00004  @file :            DufortFrankel_Scheme.cpp
00005  @description:      Implementation of the Dufort-Frankel scheme
00006  */
00007
00008  /***** Inclusion of classes *****/
00009  #include "DufortFrankel_Scheme.h"
00010  #include "ProblemDefinition.h"
00011
00012  /***** Libraries and other inclusions *****/

```

```

00013 #include <cmath>
00014 #include <vector>
00015 using namespace std;
00016
00017 /***** Methods for class DufortFrankel_Scheme *****/
00018
00019 // class constructor with parameters
00020 // to do
00021 DufortFrankel_Scheme::DufortFrankel_Scheme(const ProblemDefinition& problem)
00022 : Scheme(problem.Get_dx(), problem.Get_N(), problem.Get_dt(), problem.Get_D()), problem(problem)
00023 // use of the Scheme class, and use the object problem to avoid multiple variables
00024 {
00025     this->r = this->Get_CFL();
00026 }
00027
00028 vector<vector<double>> DufortFrankel_Scheme::dfSolution(vector<vector<double>> laasonen) {
00029     // size of the solution matrix
00030     int num_space_points = static_cast<int>(this->problem.Get_thickness() / this->problem.Get_dx()) +
00031     1;
00032     int num_time_steps = static_cast<int>(this->problem.Get_tMax() / this->problem.Get_dt()) + 1;
00033     // solution matrix with a size compose of time_index and space_index
00034     vector<vector<double>> solution(num_space_points, vector<double>(num_time_steps, 0.0));
00035     // vectors to store the different T
00036     vector<double> T_prev(num_space_points); // T_n-1
00037     vector<double> T_curr(num_space_points); // T_n
00038     vector<double> T_next(num_space_points); // T_n+1
00039     // Boundary conditions
00040     T_curr[0] = this->problem.Get_Tsur();
00041     T_curr[num_space_points-1] = this->problem.Get_Tsur();
00042     // Initialisation of T_curr
00043     for(int i=1; i<num_space_points-1; i++) {
00044         T_curr[i] = this->problem.Get_Tin();
00045     }
00046     // Storage of the initial condition
00047     for(int i=0; i<num_space_points; i++) {
00048         solution[i][0] = T_curr[i];
00049     }
00050     // Initialisation of T_prev
00051     T_prev = T_curr;
00052     // Creation of the first time_step
00053     for(int i=0; i<num_space_points; i++) {
00054         T_curr[i] = laasonen[i][1];
00055         solution[i][1] = T_curr[i];
00056     }
00057     // Loop to obtain all the other time steps
00058     for (int t=2; t<num_time_steps; t++) { // time steps
00059         // loop to obtain the result for each x
00060         for (int i=1; i<num_space_points-1; i++) { // grid steps
00061             T_next[i] = ((1.0-2.0*this->r)/(1.0+2.0*r)) * T_prev[i] + (2.0*this->r/(1.0+2.0*this->r))
00062             * (T_curr[i-1] + T_curr[i+1]);
00063         }
00064         // Storage of the current step
00065         for(int i=0; i<num_space_points; i++) {
00066             solution[i][t] = T_next[i];
00067         }
00068         // Update of the T_xxx
00069         T_prev = T_curr;
00070         T_curr = T_next;
00071     }
00072     return solution;
00073 }

```

6.7 DufortFrankel_Scheme.h File Reference

```

#include "Scheme.h"
#include "ProblemDefinition.h"
#include <iostream>
#include <vector>

```

Include dependency graph for DufortFrankel_Scheme.h: This graph shows which files directly or indirectly include this file:

Classes

- class [DufortFrankel_Scheme](#)
Implements the Dufort-Frankel explicit numerical scheme.

6.8 DufortFrankel_Scheme.h

[Go to the documentation of this file.](#)

```
00001 /* Computational Methods Assignment
00002    @author : Clémence-Philomène Hinot
00003    @date : 25/11/2025
00004    @file : DufortFrankel_Scheme.h
00005    @description: Header for the Dufort-Frankel Scheme
00006 */
00007
00008 /***** Prevention for multiple definition *****/
00009 #ifndef DUFORTFRANKEL_SCHEME_H
00010 #define DUFORTFRANKEL_SCHEME_H
00011
00012 /***** Libraries and other inclusions *****/
00013 #include "Scheme.h"
00014 #include "ProblemDefinition.h"
00015 #include <iostream>
00016 #include <vector>
00017 using namespace std;
00018
00019 /***** Declaration of class DufortFrankel_Scheme *****/
00020
00021 class DufortFrankel_Scheme : public Scheme
00022 {
00023     private:
00024         double r = 0.0;
00025         ProblemDefinition problem;
00026     public:
00027         DufortFrankel_Scheme(const ProblemDefinition& problem);
00028
00029         vector<vector<double>> dfSolution(vector<vector<double>> laasonen);
00030 };
00031
00032 /***** End of the prevention for multiple definition *****/
00033 #endif
```

6.9 Laasonen_Scheme.cpp File Reference

```
#include "Laasonen_Scheme.h"
#include "ProblemDefinition.h"
#include <iostream>
#include <vector>
#include <cmath>
```

Include dependency graph for Laasonen_Scheme.cpp:

6.10 Laasonen_Scheme.cpp

[Go to the documentation of this file.](#)

```
00001 /* Computational Methods Assignment
00002    @author : John Hoarau
00003    @date : 27/11/2025
```

```

00004      @file :          Laasonen_Scheme.cpp
00005      @description:    Implementation of the Laasonen implicit scheme.
00006  */
00007
00008  /***** Inclusion of classes *****/
00009  #include "Laasonen_Scheme.h"
00010  #include "ProblemDefinition.h"
00011  #include <iostream>
00012  #include <vector>
00013  #include <cmath>
00014
00015  /***** Libraries and other inclusions *****/
00016  using namespace std;
00017
00018  /***** Methods for class Laasonen_Scheme *****/
00019
00020  // Constructor
00021  Laasonen_Scheme::Laasonen_Scheme(const ProblemDefinition& problem)
00022      : Scheme(problem.Get_dx(), problem.Get_N(), problem.Get_dt(), problem.Get_D())
00023  {
00024      // Calculate 'r'
00025      this->r_ = (Get_D() * Get_dt()) / (Get_dx() * Get_dx());
00026
00027      // L = 31.0 cm, dx = 0.05 cm -> ~621 nodes
00028      int num_space_points = static_cast<int>(problem.Get_thickness() / problem.Get_dx()) + 1;
00029
00030      // Internal nodes are total nodes minus the two boundaries
00031      this->num_internal_nodes_ = num_space_points - 2;
00032
00033      if (num_internal_nodes_ > 0) {
00034          a_.resize(num_internal_nodes_);
00035          b_.resize(num_internal_nodes_);
00036          c_.resize(num_internal_nodes_);
00037          d_.resize(num_internal_nodes_);
00038          T_solution_.resize(num_internal_nodes_);
00039
00040          setup_thomas_coefficients();
00041      } else {
00042          std::cerr << "ERROR: Number of spatial nodes must be > 2." << std::endl;
00043      }
00044  }
00045
00046  // 1. Matrix Coefficients (Different from Crank-Nicholson)
00047  void Laasonen_Scheme::setup_thomas_coefficients() {
00048      // Equation: -r*T(i-1) + (1 + 2r)*T(i) - r*T(i+1) = T_old(i)
00049
00050      double val_a = -r_;
00051      double val_b = 1.0 + 2.0 * r_;
00052      double val_c = -r_;
00053
00054      for (int i = 0; i < num_internal_nodes_; ++i) {
00055          a_[i] = val_a;
00056          b_[i] = val_b;
00057          c_[i] = val_c;
00058      }
00059  }
00060
00061  // 2. RHS Vector (Different from Crank-Nicholson)
00062  void Laasonen_Scheme::setup_rhs_vector(const std::vector<double>& T_current, double T_sur) {
00063
00064      // General case for internal nodes: RHS is just the old temperature
00065      for (int i_int = 0; i_int < num_internal_nodes_; ++i_int) {
00066          int j_phys = i_int + 1;
00067          d_[i_int] = T_current[j_phys];
00068      }
00069
00070      // Boundary Conditions adjustment
00071      // The term -r*T(i-1) moves to RHS and becomes +r*T_sur for the first node
00072      if (num_internal_nodes_ > 0) {
00073          d_[0] += r_ * T_sur;
00074      }
00075
00076      // The term -r*T(i+1) moves to RHS and becomes +r*T_sur for the last node
00077      if (num_internal_nodes_ > 1) {
00078          d_[num_internal_nodes_ - 1] += r_ * T_sur;
00079      }
00080  }
00081
00082  // 3. Solver (Identical to Crank-Nicholson)
00083  void Laasonen_Scheme::solve_thomas_algorithm() {
00084      if (num_internal_nodes_ <= 0) return;
00085
00086      vector<double> c_prime = c_;
00087      vector<double> d_prime = d_;
00088
00089      // Forward Elimination
00090      c_prime[0] = c_prime[0] / b_[0];

```

```

00091     d_prime[0] = d_prime[0] / b_[0];
00092
00093     for (int i = 1; i < num_internal_nodes_; ++i) {
00094         double m = b_[i] - a_[i] * c_prime[i - 1];
00095         if (std::abs(m) < 1e-10) return; // Stability check
00096
00097         c_prime[i] = c_prime[i] / m;
00098         d_prime[i] = (d_prime[i] - a_[i] * d_prime[i - 1]) / m;
00099     }
00100
00101     // Backward Substitution
00102     T_solution_[num_internal_nodes_ - 1] = d_prime[num_internal_nodes_ - 1];
00103
00104     for (int i = num_internal_nodes_ - 2; i >= 0; --i) {
00105         T_solution_[i] = d_prime[i] - c_prime[i] * T_solution_[i + 1];
00106     }
00107 }
00108
00109 // 4. Main Step function
00110 void Laasonen_Scheme::solve_step(std::vector<double>& T_current, double T_sur) {
00111
00112     if (num_internal_nodes_ <= 0) return;
00113
00114     setup_rhs_vector(T_current, T_sur);
00115     solve_thomas_algorithm();
00116
00117     for (int i = 0; i < num_internal_nodes_; ++i) {
00118         T_current[i + 1] = T_solution_[i];
00119     }
00120 }
00121
00122 vector<vector<double>> Laasonen_Scheme::Solve() {
00123     int num_time_steps = static_cast<int>(Get_N()) + 1;
00124     // Calculate spatial points based on thickness and dx
00125     // Using L = 31 cm (Hardcoded here or ideally passed via a getter if added to Scheme)
00126     // Since we don't have Get_L() in Scheme, we assume the calculation below is correct relative to
the problem
00127     int num_space_points = static_cast<int>(31.0 / Get_dx()) + 1;
00128
00129     // Initialize solution matrix [space x time]
00130     vector<vector<double>> solution(num_space_points, vector<double>(num_time_steps, 0.0));
00131
00132     // Current temperature profile
00133     vector<double> T_curr(num_space_points);
00134
00135     // 1. Set Initial Conditions (t=0)
00136     for (int i = 0; i < num_space_points; ++i) {
00137         T_curr[i] = 38.0; // Tin
00138     }
00139
00140     // Boundary Conditions
00141     T_curr[0] = 149.0; // Tsur
00142     T_curr[num_space_points - 1] = 149.0;
00143
00144     // Store IC into solution column 0
00145     for (int i = 0; i < num_space_points; ++i) {
00146         solution[i][0] = T_curr[i];
00147     }
00148
00149     // 2. Time Loop
00150     for (int t = 1; t < num_time_steps; ++t) {
00151         // Perform one implicit Laasonen step (updates T_curr)
00152         solve_step(T_curr, 149.0);
00153
00154         // Store result at time t
00155         for (int i = 0; i < num_space_points; ++i) {
00156             solution[i][t] = T_curr[i];
00157         }
00158     }
00159
00160     return solution;
00161 }

```

6.11 Laasonen_Scheme.h File Reference

```

#include "Scheme.h"
#include "ProblemDefinition.h"
#include <iostream>

```

Include dependency graph for Laasonen_Scheme.h: This graph shows which files directly or indirectly include this file:

Classes

- class [Laasonen_Scheme](#)

Implements the Laasonen implicit numerical scheme.

6.12 Laasonen_Scheme.h

[Go to the documentation of this file.](#)

```

00001 /* Computational Methods Assignment
00002     @author :      John Hoarau
00003     @date :       27/11/2025
00004     @file :       Laasonen_Scheme.h
00005     @description:  Header for the Laasonen implicit scheme.
00006 */
00007
00008 /***** Prevention for mulitple definition *****/
00009 #ifndef LAASONEN_SCHEME_H
00010 #define LAASONEN_SCHEME_H
00011
00012 /***** Libraries and other inclusions *****/
00013 #include "Scheme.h"
00014 #include "ProblemDefinition.h"
00015 #include <iostream>
00016 using namespace std;
00017
00018 /***** Declaration of class Laasonen_Scheme *****/
00019
00020 class Laasonen_Scheme : public Scheme
00021 {
00022     private:
00023         double r_;
00024         int num_internal_nodes_;
00025
00026         // Vectors for the tridiagonal system (Matrix A)
00027         std::vector<double> a_;
00028         std::vector<double> b_;
00029         std::vector<double> c_;
00030
00031         // Vector for the RHS
00032         std::vector<double> d_;
00033
00034         // Solution vector
00035         std::vector<double> T_solution_;
00036
00037         void setup_thomas_coefficients();
00038
00039         void setup_rhs_vector(const std::vector<double>& T_current, double T_sur);
00040
00041         void solve_thomas_algorithm();
00042
00043     public:
00044         Laasonen_Scheme(const ProblemDefinition& problem);
00045
00046         vector<vector<double>> Solve();
00047
00048         void solve_step(std::vector<double>& T_current, double T_sur);
00049 };
00050
00051 /***** End of the prevention for mulitple definition *****/
00052 #endif // LAASONEN_SCHEME_H

```

6.13 main.cpp File Reference

```

#include "Laasonen_Scheme.h"
#include "CrankNicholson_Scheme.h"
#include "DufortFrankel_Scheme.h"
#include "Richardson_Scheme.h"
#include "Verification.h"
#include "Output.h"

```

```
#include <iostream>
#include <fstream>
#include <cmath>
Include dependency graph for main.cpp:
```

Functions

- `int main ()`
Main entry point of the application.

6.13.1 Function Documentation

6.13.1.1 main()

```
int main ()
```

Main entry point of the application.

Returns

0 on successful execution.

Definition at line 26 of file `main.cpp`.

References `ProblemDefinition::Analytic_Solution()`, `Verification::Calculation_L2_Norm()`, `Verification::Calculation_Relative_Error()`, `DufortFrankel_Scheme::dfSolution()`, `Output::Generate_Diagram()`, `Output::Generate_File()`, `Richardson_Scheme::richardsonSolution()`, `CrankNicholson_Scheme::Solve()`, `Laasonen_Scheme::Solve()`, and `Verification::Verify_Stability()`.

Here is the call graph for this function:

6.14 main.cpp

[Go to the documentation of this file.](#)

```
00001 /* Computational Methods Assignment
00002     @author :      John Hoarau, Cl  mence-Philom  ne Hinot
00003     @date  :      07/12/2025
00004     @file  :      main.cpp
00005     @brief :      Main entry point for the 1D Heat Equation Solver.
00006 */
00007
00008 #include "Laasonen_Scheme.h"
00009 #include "CrankNicholson_Scheme.h"
00010 #include "DufortFrankel_Scheme.h"
00011 #include "Richardson_Scheme.h"
00012 #include "Verification.h"
00013 #include "Output.h"
00014
00015 /***** Libraries and other inclusions *****/
00016 #include <iostream>
00017 #include <fstream>
00018 #include <cmath>
00019 using namespace std;
00020
00021 /***** Main function *****/
00026 int main()
00027 {
00028     /***** Problem Parameters *****/
00029
00030     // Temperatures
```

```

00031     double Tin = 38.0; // initial uniform temperature
00032     double Tsur = 149.0; // surface temperature (on the two sides) suddenly increased an maintained
00033
00034     // Diffusivity of the material
00035     double D = 93.0; // 93 cm^2/h
00036
00037     // Wall parameters
00038     double L = 31.0; // thickness in cm
00039
00040     // Space griding
00041     double dx = 0.05; // cm
00042
00043     double tMax = 0.5;
00044
00045     // Fourier Number
00046     int N = 100;
00047
00048     // List of time steps to test
00049     vector<double> dt_values = {0.1, 0.05, 0.025, 0.01};
00050
00051     cout << "\n===== " << endl;
00052     cout << "    STARTING MULTI-DT SIMULATION SERIES" << endl;
00053     cout << "    dt values: 0.1, 0.05, 0.025, 0.01" << endl;
00054     cout << "===== " << endl;
00055
00056     /***** Main Loop over dt *****/
00057     for (double dt : dt_values) {
00058
00059         cout << "\n##### " << endl;
00060         cout << "    RUNNING SIMULATION WITH dt = " << dt << " (N = " << N << ")" << endl;
00061         cout << "##### " << endl;
00062
00063         /***** Schemes *****/
00064
00065         // Analytical Solution
00066         ProblemDefinition wallProblem(L, dx, tMax, dt, N, Tin, Tsur, D);
00067         vector<vector<double>> analyticSolution = wallProblem.Analytic_Solution();
00068
00069         // Laasonen
00070         Laasonen_Scheme laasonen(wallProblem);
00071         vector<vector<double>> laasonenSolution = laasonen.Solve();
00072
00073         // Crank-Nicholson
00074         CrankNicholson_Scheme crankNicholson(wallProblem);
00075         vector<vector<double>> crankNicholsonSolution = crankNicholson.Solve();
00076
00077         // Dufort-Frankel
00078         DufortFrankel_Scheme dufortFrankel(wallProblem);
00079         vector<vector<double>> duforFrankelSolution = dufortFrankel.dfSolution(laasonenSolution);
00080
00081         // Richardson
00082         Richardson_Scheme richardson(wallProblem);
00083         vector<vector<double>> richardsonSolution = richardson.richardsonSolution(laasonenSolution);
00084
00085         /***** .csv file & diagram *****/
00086         visualisation.Generate_File(analyticSolution, "AnalyticSolution", dx, dt, tMax, N);
00087         // Note: Files will be overwritten (feature, not bug, for this check)
00088
00089         // Analytical Solutions
00090         visualisation.Generate_File(analyticSolution, "AnalyticSolution", dx, dt, tMax, N);
00091         visualisation.Generate_Diagram(analyticSolution, "AnalyticSolution", dx, dt, tMax, N);
00092
00093         // Laasonen
00094         visualisation.Generate_File(laasonenSolution, "Laasonen", dx, dt, tMax, N);
00095         visualisation.Generate_Diagram(laasonenSolution, "Laasonen", dx, dt, tMax, N);
00096
00097         // Crank-Nicholson
00098         visualisation.Generate_File(crankNicholsonSolution, "CrankNicholson", dx, dt, tMax, N);
00099         visualisation.Generate_Diagram(crankNicholsonSolution, "CrankNicholson", dx, dt, tMax, N);
00100
00101         // Dufort-Frankel
00102         visualisation.Generate_File(duforFrankelSolution, "DufortFrankel", dx, dt, tMax, N);
00103         visualisation.Generate_Diagram(duforFrankelSolution, "DufortFrankel", dx, dt, tMax, N);
00104
00105         // Richardson
00106         visualisation.Generate_File(richardsonSolution, "Richardson", dx, dt, tMax, N);
00107         visualisation.Generate_Diagram(richardsonSolution, "Richardson", dx, dt, tMax, N);
00108
00109         /***** Verification *****/
00110         Verification verif;
00111         int space_points = analyticSolution.size();
00112
00113         // Safety check for maximum available time index
00114         int max_time_idx = 0;
00115         if (space_points > 0) {
00116             max_time_idx = analyticSolution[0].size() - 1;
00117         }

```

```

00118
00119     double tolerance = 0.05; // 5% Relative Error Tolerance
00120     vector<double> check_times = {0.0, 0.1, 0.2, 0.3, 0.4, 0.5};
00121
00122     cout << "\n    >> VERIFICATION for dt = " << dt << " <<" << endl;
00123
00124     for (double current_t : check_times) {
00125
00126         // Calculate index: round(t / dt)
00127         int time_index = (int)round(current_t / dt);
00128
00129         cout << "\n    --- Time t = " << current_t << " (Index: " << time_index << ") ---" << endl;
00130
00131         if (time_index > max_time_idx) {
00132             cout << "    WARNING: Time index " << time_index << " exceeds available data (" <<
max_time_idx << "). Skipping." << endl;
00133             continue;
00134         }
00135
00136         // Helper to extract column for CURRENT time_index
00137         auto extract_column = [&](const vector<vector<double>&& sol) {
00138             vector<double> col(space_points);
00139             for(int i=0; i<space_points; ++i) {
00140                 if (static_cast<size_t>(time_index) < sol[i].size())
00141                     col[i] = sol[i][time_index];
00142                 else
00143                     col[i] = 0.0;
00144             }
00145             return col;
00146         };
00147
00148         vector<double> T_analytic = extract_column(analyticSolution);
00149
00150         // Helper to print results
00151         auto verify_and_print = [&](const vector<double>& T_num, string name) {
00152             double l2_norm = verif.Calculation_L2_Norm(T_num, T_analytic, dx);
00153             double rel_error = verif.Calculation_Relative_Error(T_num, T_analytic, dx);
00154
00155             cout << "    [" << name << "]" << endl;
00156
00157             if (std::isnan(rel_error) || std::isinf(rel_error))
00158                 cout << "    -> Relative Error: INF/NaN" << endl;
00159             else if (rel_error < 0)
00160                 cout << "    -> Relative Error: Error (Analytic Norm=0)" << endl;
00161             else
00162                 cout << "    -> Relative Error: " << (rel_error * 100.0) << " %" << endl;
00163
00164             bool stable = verif.Verify_Stability(rel_error, tolerance);
00165
00166             if (stable) {
00167                 cout << "    -> Status: STABLE" << endl;
00168             } else {
00169                 if (std::isnan(rel_error) || std::isinf(rel_error)) {
00170                     cout << "    -> Status: UNSTABLE (Exploded)" << endl;
00171                 } else if (rel_error < 0) {
00172                     cout << "    -> Status: UNDEFINED" << endl;
00173                 } else {
00174                     cout << "    -> Status: UNSTABLE/INACCURATE" << endl;
00175                 }
00176             }
00177         };
00178
00179         // Dufort-Frankel
00180         verify_and_print(extract_column(duforFrankelSolution), "Dufort-Frankel");
00181
00182         // Richardson
00183         verify_and_print(extract_column(richardsonSolution), "Richardson");
00184
00185         // Laasonen
00186         verify_and_print(extract_column(laasonenSolution), "Laasonen");
00187
00188         // Crank-Nicholson
00189         verify_and_print(extract_column(crankNicholsonSolution), "Crank-Nicholson");
00190     }
00191 }
00192
00193 // Proper way to finish main()
00194 return 0;
00195 }

```

6.15 Output.cpp File Reference

```
#include "Output.h"
#include <cstdio>
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <cmath>
```

Include dependency graph for Output.cpp:

6.16 Output.cpp

[Go to the documentation of this file.](#)

```
00001 /* Computational Methods Assignment
00002    @author :      Cl  mence-Philom  ne Hinot & Copilot
00003    @date :      16/10/2025
00004    @file :      Output.cpp
00005 */
00006
00007 /***** Inclusion of classes *****/
00008 #include "Output.h"
00009
00010 /***** Libraries and other inclusions *****/
00011 #include <cstdio> // for popen and pclose so use the gnuplot
00012 #include <iostream>
00013 #include <fstream>
00014 #include <string>
00015 #include <vector>
00016 #include <cmath>
00017 using namespace std;
00018
00019 /***** Methods for class ProblemDefinition *****/
00020
00021 // default class constructor
00022 Output::Output()
00023 {
00024     this->filename = "";
00025 }
00026
00027 // class constructor with parameters
00028 Output::Output(string filename)
00029 {
00030     this->filename = filename;
00031 }
00032
00033 // methods to get attributes
00034 string Output::Get_filename() const
00035 {
00036     return this->filename;
00037 }
00038
00039 // methods to set attributes
00040 void Output::Set_filename(string value_filename)
00041 {
00042     this->filename = filename;
00043 }
00044
00045 // method to generate an output file
00046 void Output::Generate_File(const vector<vector<double>>& solution, string methodName, double dx, double
    dt, double tMax, int N)
00047 {
00048     // ----- Initialisation -----
00049
00050     // calculate x coordinates
00051     vector<double> x_coordinates(N);
00052     for (int i=0; i<N; i++) {
00053         x_coordinates[i] = i * dx;
00054     }
00055
00056     // get time indices
00057     vector<int> indices;
00058     vector<double> required_times = {0.0, 0.1, 0.2, 0.3, 0.4, 0.5}; // these are the required times
    specified in the Assignment
00059     int index = 0;
```

```

00060     for (double t : required_times) { // for all the t in the range required_times
00061         if (t <= 0.5) {
00062             index = static_cast<int>(round(t / dt));
00063             indices.push_back(index);
00064         }
00065     }
00066
00067     // ----- Data exportation -----
00068
00069     // export the solution to a gnuplot format
00070     string dataFilename = methodName + ".csv";
00071     ofstream file(dataFilename);
00072     if (!file.is_open()) {
00073         cout << "Error: Cannot open file " << dataFilename << endl;
00074         return;
00075     }
00076
00077     // header
00078     file << "x(cm) ";
00079     for (double t : required_times) {
00080         file << ", " << "T(t=" << t << " ) ";
00081     }
00082     file << "\n";
00083
00084     // data rows
00085     int size_x = solution.size();
00086     for (int i = 0; i < size_x; i++) {
00087         file << x_coordinates[i] << " ";
00088         for (int idx : indices) {
00089             file << ", " << solution[i][idx];
00090         }
00091         file << "\n";
00092     }
00093
00094     file.close();
00095     cout << "Gnuplot data exported to " << dataFilename << endl;
00096 }
00097
00098 // method to visualise the solution
00099 void Output::Generate_Diagram(const vector<vector<double>& solution, string methodName, double dx,
double dt, double tMax, int N) {
00100     // ----- Initialisation -----
00101
00102     int size_x_coord = solution.size();
00103     int size_t_coord = solution[0].size();
00104
00105     vector<double> required_times = {0.0, 0.1, 0.2, 0.3, 0.4, 0.5}; // these are the required times
specified in the Assignment
00106
00107     // ----- Gnuplot script -----
00108
00109     FILE* gp = popen("gnuplot -persistent", "w");
00110     if (!gp) {
00111         std::cerr << "Error: gnuplot is not available.\n";
00112         return;
00113     }
00114
00115     // Gnuplot settings
00116     fprintf(gp, "set title '%s'\n", methodName.c_str());
00117     fprintf(gp, "set xlabel 'Position x (cm)'\n");
00118     fprintf(gp, "set ylabel 'Temperature (°C)'\n");
00119     fprintf(gp, "set grid\n");
00120     fprintf(gp, "set key outside right\n");
00121     fprintf(gp, "plot ");
00122
00123
00124
00125     // Plot commands for each time curve
00126     for (size_t i = 0; i < required_times.size(); i++) {
00127         if (i > 0) fprintf(gp, ", ");
00128         fprintf(gp, "'-' with lines title 't=%.1f hr'", required_times[i]);
00129     }
00130     fprintf(gp, "\n");
00131
00132     // Send data for each curve
00133     for (double t : required_times) {
00134         int idx = static_cast<int>(round(t / dt));
00135         if (idx >= size_t_coord) continue; // Bounds check
00136         for (int i = 0; i < size_x_coord; i++) {
00137             double x = i * dx; // dx in cm
00138             if (x > 31.0) break; // Stop at 31 cm
00139             fprintf(gp, "%f %f\n", x, solution[i][idx]);
00140         }
00141         fprintf(gp, "e\n");
00142     }
00143
00144     pclose(gp);

```

```
00145     std::cout << "Analytical solution plot generated successfully.\n";
00146 }
```

6.17 Output.h File Reference

```
#include <iostream>
#include <string>
#include <vector>
```

Include dependency graph for Output.h: This graph shows which files directly or indirectly include this file:

Classes

- class [Output](#)
Handles data export and visualization.

6.18 Output.h

[Go to the documentation of this file.](#)

```
00001 /* Computational Methods Assignment
00002  @author : John Hoarau, Cl  mence-Philom  ne Hinot
00003  @date : 16/10/2025
00004  @file : Output.h
00005 */
00006
00007 /***** Prevention for multiple definition *****/
00008 #ifndef OUTPUT_H
00009 #define OUTPUT_H
00010
00011 /***** Libraries and other inclusions *****/
00012 #include <iostream>
00013 #include <string>
00014 #include <vector>
00015 using namespace std;
00016
00017 /***** Declaration of class Output *****/
00024 class Output
00025 {
00026     private:
00027         string filename;
00028     public:
00029         Output();
00033         Output(string filename);
00039
00041         // Getters
00042         string Get_filename() const;
00043
00044         // Setters
00045         void Set_filename(string value_filename);
00046
00060         void Generate_File(const vector<vector<double>>& solution, string methodName, double dx, double
00061 dt, double tMax, int N);
00061
00074         void Generate_Diagram(const vector<vector<double>>& solution, string methodName, double dx,
00075 double dt, double tMax, int N);
00075 };
00076
00077 /***** End of the prevention for multiple definition *****/
00078 #endif
```

6.19 ProblemDefinition.cpp File Reference

```
#include "ProblemDefinition.h"
#include <cmath>
#include <vector>
```

Include dependency graph for ProblemDefinition.cpp:

Macros

- `#define M_PI 3.141592653589793`

6.19.1 Macro Definition Documentation

6.19.1.1 M_PI

```
#define M_PI 3.141592653589793
```

Definition at line 13 of file [ProblemDefinition.cpp](#).

6.20 ProblemDefinition.cpp

[Go to the documentation of this file.](#)

```
00001 /* Computational Methods Assignment
00002     @author : Clémence-Philomène Hinot
00003     @date : 16/10/2025
00004     @file : ProblemDefinition.cpp
00005 */
00006
00007 /***** Inclusion of classes *****/
00008 #include "ProblemDefinition.h"
00009
00010 /***** Libraries and other inclusions *****/
00011 #include <cmath>
00012 #ifndef M_PI
00013 #define M_PI 3.141592653589793
00014 #endif
00015 #include <vector>
00016 using namespace std;
00017
00018 /***** Methods for class ProblemDefinition *****/
00019
00020 // default class constructor
00021 ProblemDefinition::ProblemDefinition()
00022 {
00023     // coordinates
00024     this->dx = 0.0;
00025     // time
00026     this->tMax = 0.0;
00027     this->dt = 0.0;
00028     // grid space
00029     this->N = 0;
00030     // temperatures
00031     this->Tin = 0.0;
00032     this->Tsur = 0.0;
00033     // wall
00034     this->thickness = 0.0;
00035     // material
00036     this->D = 0.0;
00037 }
00038
00039 // class constructor with parameters
00040 ProblemDefinition::ProblemDefinition(double thickness, double dx, double tMax, double dt, int N,
double Tin, double Tsur, double D)
00041 {
00042     // coordinates
00043     this->dx = dx;
00044     // time
00045     this->tMax = tMax;
00046     this->dt = dt;
00047     // grid space
00048     this->N = N;
00049     // temperatures
00050     this->Tin = Tin;
00051     this->Tsur = Tsur;
00052     // wall
00053     this->thickness = thickness;
00054     // material
00055     this->D = D;
```

```

00056 }
00057
00058 // methods to get attributes
00059 double ProblemDefinition::Get_dx() const
00060 {
00061     return this->dx;
00062 }
00063
00064 double ProblemDefinition::Get_tMax() const
00065 {
00066     return this->tMax;
00067 }
00068
00069 double ProblemDefinition::Get_dt() const
00070 {
00071     return this->dt;
00072 }
00073
00074 int ProblemDefinition::Get_N() const
00075 {
00076     return this->N;
00077 }
00078
00079 double ProblemDefinition::Get_Tin() const
00080 {
00081     return this->Tin;
00082 }
00083
00084 double ProblemDefinition::Get_Tsur() const
00085 {
00086     return this->Tsur;
00087 }
00088
00089 double ProblemDefinition::Get_thickness() const
00090 {
00091     return this->thickness;
00092 }
00093
00094 double ProblemDefinition::Get_D() const
00095 {
00096     return this->D;
00097 }
00098
00099 // methods to set attributes
00100 void ProblemDefinition::Set_dx(double value)
00101 {
00102     this->dx = value;
00103 }
00104
00105 void ProblemDefinition::Set_tMax(double value)
00106 {
00107     this->tMax = value;
00108 }
00109
00110 void ProblemDefinition::Set_dt(double value)
00111 {
00112     this->dt = value;
00113 }
00114
00115 void ProblemDefinition::Set_N(int value)
00116 {
00117     this->N = value;
00118 }
00119
00120 void ProblemDefinition::Set_Tin(double value)
00121 {
00122     this->Tin = value;
00123 }
00124
00125 void ProblemDefinition::Set_Tsur(double value)
00126 {
00127     this->Tsur = value;
00128 }
00129
00130 void ProblemDefinition::Set_thickness(double value)
00131 {
00132     this->thickness = value;
00133 }
00134
00135 void ProblemDefinition::Set_D(double value)
00136 {
00137     this->D = value;
00138 }
00139
00140 // method for the analytic solution
00141 vector<vector<double>> ProblemDefinition::ProblemDefinition::Analytic_Solution()
00142 {

```

```

00143 // size of the vector result
00144 int size_x = static_cast<int>(Get_thickness() / Get_dx()) + 1; // +1 to include endpoint
00145 int size_t = static_cast<int>(Get_tMax() / Get_dt()) + 1; // +1 to include endpoint
00146
00147 // vector for the storage of the result
00148 vector<vector<double>> analytic_result(size_x, vector<double>(size_t, 0.0));
00149
00150 // initialisation of variables
00151 double x = 0.0;
00152 double t = 0.0;
00153 double sum = 0.0;
00154
00155 // first loop to store the result for each x position
00156 for (int i=0; i<size_x; i++) {
00157     t = 0.0; // initialisation of the t
00158
00159     // second loop to calculate the sum for each t
00160     for (int j=0; j<size_t; j++) {
00161         sum = 0.0; // initialisation of the sum
00162
00163         // third loop to calculate the sum
00164         for (int k=1; k<=Get_N(); k++) {
00165             sum += exp(- Get_D() * pow((k*M_PI / Get_thickness()), 2) * t) * ((1-pow(-1,
00166 k))/(k*M_PI)) * sin(k*M_PI*x/Get_thickness());
00167         }
00168         analytic_result[i][j] = Get_Tsur() + 2 * (Get_Tin() - Get_Tsur()) * sum;
00169
00170         t += Get_dt(); // increment the time
00171     }
00172
00173     x += Get_dx(); // increment the position
00174 }
00175
00176 return analytic_result;
00177 }

```

6.21 ProblemDefinition.h File Reference

```
#include <iostream>
```

```
#include <vector>
```

Include dependency graph for ProblemDefinition.h: This graph shows which files directly or indirectly include this file:

Classes

- class [ProblemDefinition](#)

6.22 ProblemDefinition.h

[Go to the documentation of this file.](#)

```

00001 /* Computational Methods Assignment
00002 @author : Clémence-Philomène Hinot
00003 @date : 16/10/2025
00004 @file : ProblemDefinition.h
00005 */
00006
00007 /***** Prevention for mulitple definition *****/
00008 #ifndef PROBLEMDEFINITION_H
00009 #define PROBLEMDEFINITION_H
00010
00011 /***** Libraries and other inclusions *****/
00012 #include <iostream>
00013 #include <vector>
00014 using namespace std;
00015
00016 /***** Declaration of class ProblemDefinition *****/
00017 class ProblemDefinition
00018 {

```

```

00019     private:
00020     private:
00021         double dx = 0.0;
00022         double tMax = 0.0;
00023         double dt = 0.0;
00024         int N = 0;
00025         double Tin = 0.0;
00026         double Tsur = 0.0;
00027         double thickness = 0.0;
00028         double D = 0.0;
00029
00030     public:
00031         ProblemDefinition();
00032
00033         ProblemDefinition(double thickness, double dx, double tMax, double dt, int N, double Tin,
00034 double Tsur, double D);
00035
00036         double Get_dx() const;
00037
00038         double Get_tMax() const;
00039
00040         double Get_dt() const;
00041
00042         int Get_N() const;
00043
00044         double Get_Tin() const;
00045
00046         double Get_Tsur() const;
00047
00048         double Get_thickness() const;
00049
00050         double Get_D() const;
00051
00052         void Set_dx(double value);
00053
00054         void Set_tMax(double value);
00055
00056         void Set_dt(double value);
00057
00058         void Set_N(int value);
00059
00060         void Set_Tin(double value);
00061
00062         void Set_Tsur(double value);
00063
00064         void Set_thickness(double value);
00065
00066         void Set_D(double value);
00067
00068         vector<vector<double>> Analytic_Solution();
00069     };
00070
00071 /***** End of the prevention for multiple definition *****/
00072 #endif

```

6.23 README.md File Reference

6.24 Richardson_Scheme.cpp File Reference

```

#include "Richardson_Scheme.h"
#include "ProblemDefinition.h"
#include <vector>
#include <cmath>

```

Include dependency graph for Richardson_Scheme.cpp:

6.25 Richardson_Scheme.cpp

[Go to the documentation of this file.](#)

```

00001 /* Computational Methods Assignment
00002 @author : Cl  mence-Philom  ne Hinot

```

```

00003      @date :          16/10/2025
00004      @file :          Richardson_Scheme.cpp
00005      @description:     Implementation of the Richardson scheme
00006  */
00007
00008  /***** Inclusion of classes *****/
00009  #include "Richardson_Scheme.h"
00010  #include "ProblemDefinition.h"
00011
00012  /***** Libraries and other inclusions *****/
00013  #include <vector>
00014  #include <cmath>
00015  using namespace std;
00016
00017  /***** Methods for class Richardson_Scheme *****/
00018
00019  // class constructor with parameters
00020  Richardson_Scheme::Richardson_Scheme(const ProblemDefinition& problem)
00021  : Scheme(problem.Get_dx(), problem.Get_N(), problem.Get_dt(), problem.Get_D()), problem(problem) //
    use the class "Scheme" and store the problem
00022  {
00023      // CFL number
00024      this->r = this->Get_CFL();
00025  }
00026
00027  // methods
00028  vector<vector<double>> Richardson_Scheme::richardsonSolution(vector<vector<double>> laasonen) {
00029      // size of the solution matrix
00030      int num_time_steps = static_cast<int>(this->problem.Get_tMax() / this->problem.Get_dt()) + 1;
00031      int num_space_points = static_cast<int>(this->problem.Get_thickness() / this->problem.Get_dx())
00032      +1;
00033
00034      // solution matrix with a size compose of time_index and space_index
00035      vector<vector<double>> solution(num_space_points, vector<double>(num_time_steps, 0.0));
00036
00037      // vectors to store the different T
00038      vector<double> T_prev(num_space_points); // T_{n-1}
00039      vector<double> T_curr(num_space_points); // T_n
00040      vector<double> T_next(num_space_points); // T_{n+1}
00041
00042      // Boundary conditions
00043      T_curr[0] = this->problem.Get_Tsur();
00044      T_curr[num_space_points-1] = this->problem.Get_Tsur();
00045
00046      // Initialisation of T_curr
00047      for(int i=1; i<num_space_points-1; i++) {
00048          T_curr[i] = this->problem.Get_Tin();
00049      }
00050
00051      // Storage of the initial condition
00052      for(int i=0; i<num_space_points; i++) {
00053          solution[i][0] = T_curr[i];
00054      }
00055
00056      // Initialisation of T_prev
00057      T_prev = T_curr;
00058
00059      // Creation of the first time_step
00060      for(int i=0; i<num_space_points; i++) {
00061          T_curr[i] = laasonen[i][1];
00062          solution[i][1] = T_curr[i];
00063      }
00064
00065      // Loop to obtain all the other time steps
00066      for (int t=2; t<num_time_steps; t++) { // time steps
00067          // loop to obtain the result for each x
00068          for (int i=1; i<num_space_points-1; i++) { // grid steps
00069              T_next[i] = T_prev[i] + 2.0 * this->r * (T_curr[i+1] - 2.0 * T_curr[i] + T_curr[i-1]);
00070          }
00071
00072          // Storage of the current step
00073          for(int i=0; i<num_space_points; i++) {
00074              solution[i][t] = T_next[i];
00075          }
00076
00077          // Update of the T_xxx
00078          T_prev = T_curr;
00079          T_curr = T_next;
00080      }
00081      return solution;
00082  }

```

6.26 Richardson_Scheme.h File Reference

```
#include "Scheme.h"
#include "ProblemDefinition.h"
#include <iostream>
#include <vector>
```

Include dependency graph for Richardson_Scheme.h: This graph shows which files directly or indirectly include this file:

Classes

- class [Richardson_Scheme](#)
Implements the Richardson explicit numerical scheme.

6.27 Richardson_Scheme.h

[Go to the documentation of this file.](#)

```
00001 /* Computational Methods Assignment
00002    @author :      John Hoarau, Clémence-Philomène Hinot
00003    @date :        16/10/2025
00004    @file :        Richardson_Scheme.h
00005    @description:   Header for the Richardson Scheme
00006 */
00007
00008 /***** Prevention for multiple definition *****/
00009 #ifndef RICHARDSON_SCHEME_H
00010 #define RICHARDSON_SCHEME_H
00011
00012 /***** Libraries and other inclusions *****/
00013 #include "Scheme.h"
00014 #include "ProblemDefinition.h"
00015 #include <iostream>
00016 #include <vector>
00017 using namespace std;
00018
00019 /***** Declaration of class Richardson_Scheme *****/
00020
00028 class Richardson_Scheme : public Scheme
00029 {
00030     private:
00031         double r = 0.0;
00032         ProblemDefinition problem;
00033     public:
00038         Richardson_Scheme(const ProblemDefinition& problem);
00039
00045         vector<vector<double>> richardsonSolution(vector<vector<double>> laasonen);
00046 };
00047
00048 /***** End of the prevention for multiple definition *****/
00049 #endif
```

6.28 Scheme.cpp File Reference

```
#include "Scheme.h"
#include <cmath>
```

Include dependency graph for Scheme.cpp:

6.29 Scheme.cpp

[Go to the documentation of this file.](#)

```

00001 /* Computational Methods Assignment
00002    @author :      John Hoarau, Cl  mence-Philom  ne Hinot
00003    @date :       16/10/2025
00004    @file :       Scheme.cpp
00005 */
00006
00007 /***** Inclusion of classes *****/
00008 #include "Scheme.h"
00009
00010 /***** Libraries and other inclusions *****/
00011 #include <cmath>
00012 using namespace std;
00013
00014 /***** Methods for class ProblemDefinition *****/
00015
00016 // default class constructor
00017 Scheme::Scheme()
00018 {
00019     this->dx = 0.0;
00020     this->N = 0;
00021     this->dt = 0.0;
00022     this->D = 0.0;
00023     this->r = 0.0;
00024 }
00025
00026 // class constructor with parameters
00027 Scheme::Scheme(double dx, int N, double dt, double D)
00028 {
00029     this->dx = dx;
00030     this->N = N;
00031     this->dt = dt;
00032     this->D = D;
00033 }
00034
00035 // methods to get attributes
00036 double Scheme::Get_dx() const
00037 {
00038     return this->dx;
00039 }
00040
00041 int Scheme::Get_N() const
00042 {
00043     return this->N;
00044 }
00045
00046 double Scheme::Get_dt() const
00047 {
00048     return this->dt;
00049 }
00050
00051 double Scheme::Get_D() const
00052 {
00053     return this->D;
00054 }
00055
00056 // methods to set attributes
00057 void Scheme::Set_dx(double value)
00058 {
00059     this->dx = value;
00060 }
00061
00062 void Scheme::Set_N(int value)
00063 {
00064     this->N = value;
00065 }
00066
00067 void Scheme::Set_dt(double value)
00068 {
00069     this->dt = value;
00070 }
00071
00072 void Scheme::Set_D(double value)
00073 {
00074     this->D = value;
00075 }
00076
00077 // method to obtain the CFL number
00078
00079 double Scheme::Get_CFL() {
00080     this->r = this->D * this->dt / pow(this->dx, 2);
00081     return this->r;
00082 }
```

6.30 Scheme.h File Reference

```
#include <iostream>
```

Include dependency graph for Scheme.h: This graph shows which files directly or indirectly include this file:

Classes

- class [Scheme](#)
Base class for numerical schemes.

6.31 Scheme.h

[Go to the documentation of this file.](#)

```
00001 /* Computational Methods Assignment
00002  @author : John Hoarau, Cl  mence-Philom  ne Hinot
00003  @date : 16/10/2025
00004  @file : Scheme.h
00005 */
00006
00007 /***** Prevention for multiple definition *****/
00008 #ifndef SCHEME_H
00009 #define SCHEME_H
00010
00011 /***** Libraries and other inclusions *****/
00012 #include <iostream>
00013 using namespace std;
00014
00015 /***** Declaration of class Scheme *****/
00023 class Scheme
00024 {
00025     private:
00026         double dx;
00027         int N;
00028         double dt;
00029         double D;
00030         double r;
00031
00032     public:
00036         Scheme();
00037
00045         Scheme(double dx, int N, double dt, double D);
00046
00047         // Getters
00048         double Get_dx() const;
00049         int Get_N() const;
00050         double Get_dt() const;
00051         double Get_D() const;
00052
00053         // Setters
00054         void Set_dx(double value);
00055         void Set_N(int value);
00056         void Set_dt(double value);
00057         void Set_D(double value);
00058
00063         double Get_CFL();
00064 };
00065
00066 /***** End of the prevention for multiple definition *****/
00067 #endif
```

6.32 Verification.cpp File Reference

```
#include "Verification.h"
#include <cmath>
#include <vector>
#include <iostream>
#include <string>
```

Include dependency graph for Verification.cpp:

6.33 Verification.cpp

[Go to the documentation of this file.](#)

```

00001 /* Computational Methods Assignment
00002     @author :      John Hoarau, Github copilot
00003     @date  :      16/10/2025
00004     @file  :      Verification.cpp
00005     @description:  This class provides methods for verifying numerical results and conditions.
00006 */
00007
00008 /***** Inclusion of classes *****/
00009 #include "Verification.h"
00010
00011 /***** Libraries and other inclusions *****/
00012 #include <cmath>
00013 #include <vector>
00014 #include <iostream> // Added for cout and cerr
00015 #include <string>    // Added for string type
00016 using namespace std;
00017
00018 /***** Methods for class Verification *****/
00019
00020 // default class constructor
00021 Verification::Verification()
00022 {
00023     this->CFL = 0.0;
00024 }
00025
00026 // class constructor with parameters
00027 Verification::Verification(double CFL)
00028 {
00029     this->CFL = CFL;
00030 }
00031
00032 // method to get attributes
00033 double Verification::Get_CFL() const
00034 {
00035     return this->CFL;
00036 }
00037
00038 // method to set attributes
00039 void Verification::Set_CFL(double value)
00040 {
00041     this->CFL = value;
00042 }
00043
00044 // methods to do the verifications
00045 // methods to do the verifications
00046 bool Verification::Verify_Stability(double error, double tolerance)
00047 {
00048     // Check for NaN or Infinity (Explosion)
00049     if (std::isnan(error) || std::isinf(error)) {
00050         return false;
00051     }
00052
00053     // Check against tolerance
00054     if (error > tolerance) {
00055         return false;
00056     }
00057
00058     // Otherwise, stable
00059     return true;
00060 }
00061
00062 double Verification::Calculation_L2_Norm(const vector<double>& numerical, const vector<double>&
analytical, double dx)
00063 {
00064     double sum_sq_diff = 0.0;
00065     size_t n = numerical.size();
00066
00067     if (n != analytical.size()) {
00068         cerr << "Error: Vector sizes do not match for Norm calculation." << endl;
00069         return -1.0;
00070     }
00071
00072     for (size_t i = 0; i < n; ++i) {
00073         double diff = numerical[i] - analytical[i];
00074         sum_sq_diff += diff * diff;
00075     }
00076
00077     // Correct L2 Norm definition: sqrt( integral |u|^2 dx ) approx sqrt( sum * dx )
00078     return sqrt(sum_sq_diff * dx);
00079 }
00080

```

```

00081 double Verification::Calculation_Relative_Error(const vector<double>& numerical, const vector<double>&
analytical, double dx)
00082 {
00083     // Calculate Absolute L2 Error (Numerator)
00084     double abs_error = Calculation_L2_Norm(numerical, analytical, dx);
00085     if (abs_error < 0) return -1.0; // Error in calculation
00086
00087     // Calculate L2 Norm of Analytical Solution (Denominator)
00088     // We pass a zero vector as the "second" vector to calculate the norm of 'analytical' directly
00089     // Or just re-implement the sum squares for 'analytical'
00090
00091     double sum_sq_ana = 0.0;
00092     // size_t n = analytical.size(); // Unused
00093     for (double val : analytical) {
00094         sum_sq_ana += val * val;
00095     }
00096     double ana_norm = sqrt(sum_sq_ana * dx);
00097
00098     if (ana_norm == 0.0) {
00099         cerr << "Error: Analytical solution norm is zero, cannot calculate relative error." << endl;
00100         return -1.0;
00101     }
00102
00103     return abs_error / ana_norm;
00104 }

```

6.34 Verification.h File Reference

```

#include <iostream>
#include <vector>
#include <string>

```

Include dependency graph for Verification.h: This graph shows which files directly or indirectly include this file:

Classes

- class [Verification](#)
Provides methods for verifying numerical results and conditions.

6.35 Verification.h

[Go to the documentation of this file.](#)

```

00001 /* Computational Methods Assignment
00002    @author :      John Hoarau, Github copilot
00003    @date :       16/10/2025
00004    @file :       Verification.h
00005    @description:  Header for verification functions and utilities.
00006 */
00007
00008 /***** Prevention for multiple definition *****/
00009 #ifndef VERIFICATION_H
00010 #define VERIFICATION_H
00011
00012 /***** Libraries and other inclusions *****/
00013 #include <iostream>
00014 #include <vector>
00015 #include <string> // Added for string type
00016 using namespace std;
00017
00018 /***** Declaration of class Verification *****/
00026 class Verification
00027 {
00028     private:
00029         double CFL;
00030     public:
00034         Verification();
00035
00040         Verification(double CFL);
00041

```

```
00042         // Getters
00043         double Get_CFL() const;
00044
00045         // Setters
00046         void Set_CFL(double value);
00047
00055         bool Verify_Stability(double error, double tolerance);
00056
00067         double Calculation_L2_Norm(const vector<double>& numerical, const vector<double>& analytical,
double dx);
00068
00079         double Calculation_Relative_Error(const vector<double>& numerical, const vector<double>&
analytical, double dx);
00080 };
00081
00082 /***** End of the prevention for mulitple definition *****/
00083 #endif
```

Index

1D Heat Equation Solver - Cranfield Assignment, [1](#)

a_

CrankNicholson_Scheme, [14](#)

Laasonen_Scheme, [21](#)

Analytic_Solution

ProblemDefinition, [28](#)

b_

CrankNicholson_Scheme, [14](#)

Laasonen_Scheme, [21](#)

c_

CrankNicholson_Scheme, [14](#)

Laasonen_Scheme, [21](#)

Calculation_L2_Norm

Verification, [46](#)

Calculation_Relative_Error

Verification, [46](#)

CFL

Verification, [48](#)

CrankNicholson_Scheme, [9](#)

a_, [14](#)

b_, [14](#)

c_, [14](#)

CrankNicholson_Scheme, [11](#)

d_, [14](#)

num_internal_nodes_, [14](#)

r_, [14](#)

setup_rhs_vector, [12](#)

setup_thomas_coefficients, [12](#)

Solve, [12](#)

solve_step, [13](#)

solve_thomas_algorithm, [13](#)

T_solution_, [15](#)

CrankNicholson_Scheme.cpp, [49](#)

CrankNicholson_Scheme.h, [51](#)

D

ProblemDefinition, [34](#)

Scheme, [43](#)

d_

CrankNicholson_Scheme, [14](#)

Laasonen_Scheme, [22](#)

dfSolution

DufortFrankel_Scheme, [17](#)

dt

ProblemDefinition, [34](#)

Scheme, [43](#)

DufortFrankel_Scheme, [15](#)

dfSolution, [17](#)

DufortFrankel_Scheme, [16](#)

problem, [17](#)

r, [17](#)

DufortFrankel_Scheme.cpp, [52](#)

DufortFrankel_Scheme.h, [53](#)

dx

ProblemDefinition, [34](#)

Scheme, [43](#)

filename

Output, [25](#)

Generate_Diagram

Output, [24](#)

Generate_File

Output, [24](#)

Get_CFL

Scheme, [41](#)

Verification, [47](#)

Get_D

ProblemDefinition, [28](#)

Scheme, [41](#)

Get_dt

ProblemDefinition, [28](#)

Scheme, [41](#)

Get_dx

ProblemDefinition, [29](#)

Scheme, [42](#)

Get_filename

Output, [25](#)

Get_N

ProblemDefinition, [29](#)

Scheme, [42](#)

Get_thickness

ProblemDefinition, [29](#)

Get_Tin

ProblemDefinition, [30](#)

Get_tMax

ProblemDefinition, [30](#)

Get_Tsur

ProblemDefinition, [30](#)

Laasonen_Scheme, [18](#)

a_, [21](#)

b_, [21](#)

c_, [21](#)

d_, [22](#)

Laasonen_Scheme, [19](#)

num_internal_nodes_, [22](#)

- [r_](#), [22](#)
 - [setup_rhs_vector](#), [20](#)
 - [setup_thomas_coefficients](#), [20](#)
 - [Solve](#), [20](#)
 - [solve_step](#), [20](#)
 - [solve_thomas_algorithm](#), [21](#)
 - [T_solution_](#), [22](#)
- [Laasonen_Scheme.cpp](#), [54](#)
- [Laasonen_Scheme.h](#), [56](#)
- [M_PI](#)
 - [ProblemDefinition.cpp](#), [64](#)
- [main](#)
 - [main.cpp](#), [58](#)
- [main.cpp](#), [57](#)
- [main](#), [58](#)
- [N](#)
 - [ProblemDefinition](#), [34](#)
 - [Scheme](#), [44](#)
- [num_internal_nodes_](#)
 - [CrankNicholson_Scheme](#), [14](#)
 - [Laasonen_Scheme](#), [22](#)
- [Output](#), [23](#)
 - [filename](#), [25](#)
 - [Generate_Diagram](#), [24](#)
 - [Generate_File](#), [24](#)
 - [Get_filename](#), [25](#)
 - [Output](#), [23](#)
 - [Set_filename](#), [25](#)
- [Output.cpp](#), [61](#)
- [Output.h](#), [63](#)
- [problem](#)
 - [DufortFrankel_Scheme](#), [17](#)
 - [Richardson_Scheme](#), [38](#)
- [ProblemDefinition](#), [26](#)
 - [Analytic_Solution](#), [28](#)
 - [D](#), [34](#)
 - [dt](#), [34](#)
 - [dx](#), [34](#)
 - [Get_D](#), [28](#)
 - [Get_dt](#), [28](#)
 - [Get_dx](#), [29](#)
 - [Get_N](#), [29](#)
 - [Get_thickness](#), [29](#)
 - [Get_Tin](#), [30](#)
 - [Get_tMax](#), [30](#)
 - [Get_Tsur](#), [30](#)
 - [N](#), [34](#)
 - [ProblemDefinition](#), [27](#)
 - [Set_D](#), [30](#)
 - [Set_dt](#), [31](#)
 - [Set_dx](#), [31](#)
 - [Set_N](#), [31](#)
 - [Set_thickness](#), [32](#)
 - [Set_Tin](#), [32](#)
 - [Set_tMax](#), [32](#)
 - [Set_Tsur](#), [32](#)
 - [thickness](#), [34](#)
 - [Tin](#), [35](#)
 - [tMax](#), [35](#)
 - [Tsur](#), [35](#)
- [ProblemDefinition.cpp](#), [63](#)
 - [M_PI](#), [64](#)
- [ProblemDefinition.h](#), [66](#)
- [r](#)
 - [DufortFrankel_Scheme](#), [17](#)
 - [Richardson_Scheme](#), [38](#)
 - [Scheme](#), [44](#)
- [r_](#)
 - [CrankNicholson_Scheme](#), [14](#)
 - [Laasonen_Scheme](#), [22](#)
- [README.md](#), [67](#)
- [Richardson_Scheme](#), [35](#)
 - [problem](#), [38](#)
 - [r](#), [38](#)
 - [Richardson_Scheme](#), [37](#)
 - [richardsonSolution](#), [38](#)
- [Richardson_Scheme.cpp](#), [67](#)
- [Richardson_Scheme.h](#), [69](#)
- [richardsonSolution](#)
 - [Richardson_Scheme](#), [38](#)
- [Scheme](#), [39](#)
 - [D](#), [43](#)
 - [dt](#), [43](#)
 - [dx](#), [43](#)
 - [Get_CFL](#), [41](#)
 - [Get_D](#), [41](#)
 - [Get_dt](#), [41](#)
 - [Get_dx](#), [42](#)
 - [Get_N](#), [42](#)
 - [N](#), [44](#)
 - [r](#), [44](#)
 - [Scheme](#), [40](#)
 - [Set_D](#), [42](#)
 - [Set_dt](#), [42](#)
 - [Set_dx](#), [43](#)
 - [Set_N](#), [43](#)
- [Scheme.cpp](#), [69](#)
- [Scheme.h](#), [71](#)
- [Set_CFL](#)
 - [Verification](#), [47](#)
- [Set_D](#)
 - [ProblemDefinition](#), [30](#)
 - [Scheme](#), [42](#)
- [Set_dt](#)
 - [ProblemDefinition](#), [31](#)
 - [Scheme](#), [42](#)
- [Set_dx](#)
 - [ProblemDefinition](#), [31](#)
 - [Scheme](#), [43](#)
- [Set_filename](#)
 - [Output](#), [25](#)
- [Set_N](#)

- ProblemDefinition, [31](#)
 - Scheme, [43](#)
- Set_thickness
 - ProblemDefinition, [32](#)
- Set_Tin
 - ProblemDefinition, [32](#)
- Set_tMax
 - ProblemDefinition, [32](#)
- Set_Tsur
 - ProblemDefinition, [32](#)
- setup_rhs_vector
 - CrankNicholson_Scheme, [12](#)
 - Laasonen_Scheme, [20](#)
- setup_thomas_coefficients
 - CrankNicholson_Scheme, [12](#)
 - Laasonen_Scheme, [20](#)
- Solve
 - CrankNicholson_Scheme, [12](#)
 - Laasonen_Scheme, [20](#)
- solve_step
 - CrankNicholson_Scheme, [13](#)
 - Laasonen_Scheme, [20](#)
- solve_thomas_algorithm
 - CrankNicholson_Scheme, [13](#)
 - Laasonen_Scheme, [21](#)
- T_solution_
 - CrankNicholson_Scheme, [15](#)
 - Laasonen_Scheme, [22](#)
- thickness
 - ProblemDefinition, [34](#)
- Tin
 - ProblemDefinition, [35](#)
- tMax
 - ProblemDefinition, [35](#)
- Tsur
 - ProblemDefinition, [35](#)
- Verification, [44](#)
 - Calculation_L2_Norm, [46](#)
 - Calculation_Relative_Error, [46](#)
 - CFL, [48](#)
 - Get_CFL, [47](#)
 - Set_CFL, [47](#)
 - Verification, [45](#)
 - Verify_Stability, [47](#)
- Verification.cpp, [71](#)
- Verification.h, [73](#)
- Verify_Stability
 - Verification, [47](#)